The Algebra of Intervention Fields

Compositional Denotational, Operational, and Statistical Semantics for RAG Optimization

RAG-MATHS Architecture Study

August 2026

Abstract

Retrieval-augmented generation optimization is usually represented as a flat search over configuration values. That representation is inadequate for a production RAG system. A candidate can alter source admission, chunk identity, derived representations, index bytes, approximate search error, query interpretation, remote disclosure, context construction, agent trajectories, latency behavior, and frontend state. Its cost and meaning depend on where it acts, which downstream artifacts it invalidates, which observations it is expected to preserve, and which evidence must be collected before promotion.

This thesis develops an abstract mathematical backbone for a composable optimization architecture. The central construction is an **optimization field**: an indexed family of categories whose objects are behavior-complete system specifications and whose morphisms are typed interventions. The fibers are graded by semantic effect, equipped with local optics, change actions, dependency derivatives, statistical experiment functors, and constraint-first decision relations. Open RAG behavior is interpreted as a parameterized stateful morphism in a Markov category; runtime campaigns are given a small-step event semantics and a denotational fold. Multi-fidelity evaluation is analyzed through comparison of statistical experiments, and artifact reuse is justified by a support-disjointness theorem rather than content identity alone.

A self-contained standard-library Go implementation accompanies the thesis. It provides finite Markov kernels, lawful lenses, change actions, dependency closure, typed intervention spaces, paired experiments, a finite Blackwell witness, mergeable statistics, interval-aware gates, Pareto fronts, an event-sourced campaign reducer, finite state-space exploration, a TLA+ model target, and a hybrid lexical/vector RAG optimizer. The executable campaign evaluates 575 candidate specifications at low fidelity, advances 56 and then 14, identifies six passing Pareto points, and promotes one candidate through an auditable reducer. The implementation is an executable model of the theory, not a claim that finite tests prove all production properties.

Table of Contents

Preface

This volume expands the optimization-field proposal introduced in Chapter 21 of *The Semantics and Dynamics of Retrieval-Augmented Systems*. The earlier chapter made three practical claims: a RAG candidate is a controlled intervention rather than an arbitrary configuration; its dependency closure determines rebuild and reuse; and promotion is a constrained, multi-objective judgment rather than maximization of one score. Those claims were architecturally useful but mathematically underdeveloped. They did not yet explain what kind of object an intervention is, how interventions compose, how a local change acts on a whole release, what it means for behavior to be preserved, how runtime observations relate to denotation, or how a generic optimizer can remain ignorant of RAG internals while still validating RAG-specific work.

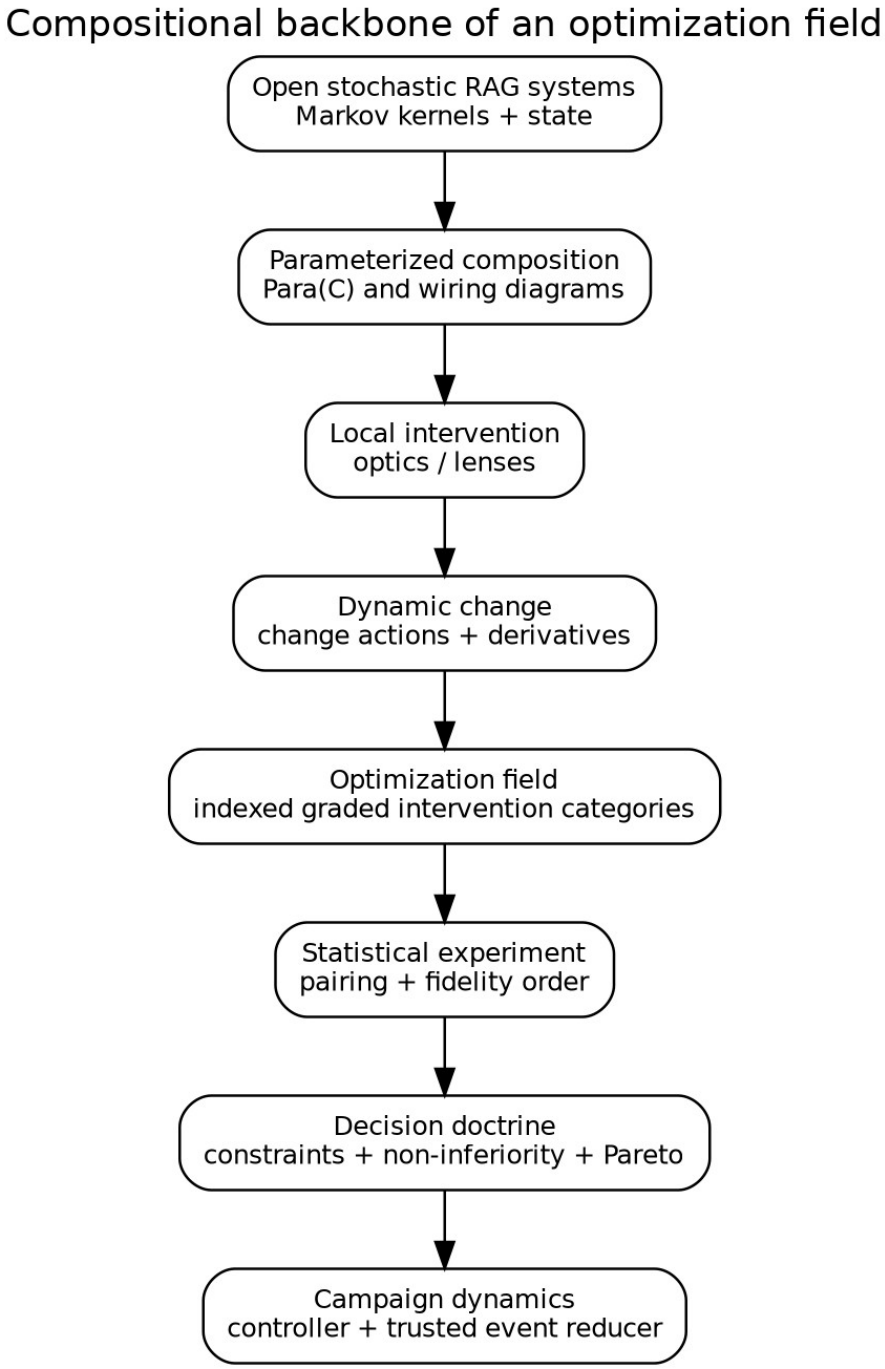
The present thesis supplies that backbone. It is deliberately positioned between pure category theory and production systems engineering. It uses category-theoretic structures because they expose interfaces and laws that ordinary configuration schemas hide. It uses operational semantics because an optimizer is a stateful runtime system with retries, partial evidence, cancellations, and promotion authority. It uses statistical decision theory because the behavior of a RAG release is stochastic and finite evidence cannot justify exact equality. It uses incremental computation because changes to indexing and querying induce structured rebuild and reevaluation work. None of these perspectives is sufficient alone.

The result is not a universal theorem that every possible RAG optimizer must instantiate exactly. It is a proposed semantic architecture with explicit assumptions. Definitions and propositions are separated from engineering conventions. Proof sketches state the conditions on which they depend. Executable checks validate finite instances of laws; they do not replace general proof. Product security, user impact, and provider behavior remain product-specific obligations.

Principal contributions

The thesis makes nine connected contributions.

1. It defines a production RAG release as an **open, stateful, stochastic, parameterized system** rather than a static retrieval function.
2. It defines a candidate as a **morphism out of a baseline**. Candidate identity therefore includes causal path and declared intervention, not only endpoint bytes.
3. It uses **optics and lawful lenses** to represent local parameter focus and reconstruction of behavior-complete specifications.
4. It uses **change actions and derivatives** to give semantic meaning to invalidation, incremental rebuild, cache reuse, and experiment reuse.
5. It defines an **optimization field** as an indexed or fibred family of intervention categories over system architectures.
6. It grades interventions by a join-semilattice of semantic effects and derives evidence obligations through a monotone map.
7. It interprets evaluation as a **statistical experiment**, exact pairing as copying deterministic experimental context, and nested fidelity as a Blackwell garbling relation when such a witness exists.
8. It separates search allocation from promotion, using hard constraints, interval-valued non-inferiority, and Pareto order instead of a promotion scalar.
9. It supplies a self-contained implementation and migration path for the existing `ragkit` and `ragopt` packages.



The mathematical backbone proceeds from open stochastic behavior to a validated campaign controller.

How to read this volume

Readers primarily concerned with architecture can begin with Chapters 1, 5, 15, 25, 28, and 31. Readers concerned with formal semantics should read Parts II and III in order. Readers implementing `ragopt` can begin with Chapters 22 through 30 and the API appendices. The sandbox is described in Part VI; its source tree is included beside the manuscript.

Notation is introduced locally and summarized in Appendix A. The base category of stochastic maps is written K; an architecture is A; its specification space is Θ_A; a baseline is θ_0; and an intervention is $i : \theta_0 \to \theta_1$. Composition is written $j \circ i$, with i performed first. The semantic effect grade of i is $g(i)$. Its changed support is $\mathrm{supp}(i)$ and its downstream impact closure is $\mathrm{cl}_A(\mathrm{supp}(i))$.

Part I. The optimization problem reconstructed

1. Why a flat search space is the wrong object

1.1 The conventional representation

A conventional optimizer begins with a product space:

$$\Theta = \Theta_{\text{chunk}} \times \Theta_{\text{embed}} \times \Theta_{\text{index}} \times \Theta_{\text{query}} \times \Theta_{\text{rank}} \times \Theta_{\text{answer}}.$$

A trial samples $\theta \in \Theta$, builds whatever is necessary, evaluates a scalar objective $J(\theta)$, and updates a search policy. This representation is useful for an isolated model whose parameters have uniform status. It is misleading for a production RAG service.

The coordinates do not have uniform semantics. Changing worker count may preserve all successful outcomes until a deadline is crossed. Changing HNSW search breadth changes an approximation relation to an exact oracle. Changing chunk boundaries changes evidence identity and may change what facts are representable. Changing authorization order changes disclosure even if final answer text is unchanged. Changing a widget projection changes user-visible state without changing retrieval. Treating all of these as values in one untyped product erases the reasons they require different evidence.

The coordinates also do not have uniform operational cost. A fusion weight can often reuse channel rankings. A reranker pool change can reuse a retained fused prefix. A chunker change invalidates chunks, derived representations, embeddings, index bytes, query traces, contexts, and answer observations. A provider timeout may invalidate only runtime measurements unless it changes fallback frequency. The optimizer must know the *support* of a change and the transitive dependency relation, not just the before and after JSON documents.

Finally, endpoint equality is insufficient. Suppose two paths reach the same final release:

$$\theta_0 \xrightarrow{i} \theta_a \xrightarrow{j} \theta_1, \theta_0 \xrightarrow{k} \theta_b \xrightarrow{\ell} \theta_1.$$

The endpoint bytes may agree while the causal hypotheses, intermediate evidence, reused artifacts, and audit records differ. A system intended to learn from interventions must retain the arrow, not only its codomain.

1.2 The field metaphor

The word *field* is used in a structural, not physical, sense. Over every system architecture lies a family of valid specifications, interventions, observations, and decisions. Moving to a different architecture changes which parameters exist and how evidence can be transported. Within one architecture, local changes compose. Across architectures, restriction or translation maps carry a specification and its support into another context.

This suggests an indexed construction:

$$Opt : A^{op} \to Cat,$$

where A is a category of architectures and $Opt(A)$ is the intervention category for architecture A. The associated Grothendieck construction collects all fibers into one total category while retaining the projection to architecture. The term *optimization field* refers to this indexed family together with its behavioral semantics, change calculus, experiment doctrine, and decision doctrine.

1.3 Requirements for the replacement

A replacement for a flat search space must support all of the following without making the generic optimizer a RAG framework:

- local, typed, law-governed changes;
- sequential composition and certified parallel composition;
- behavior-complete release identity;
- stochastic and stateful runtime interpretation;
- explicit observational equivalence and approximation budgets;
- dependency-aware rebuild and cache reuse;
- effect-sensitive evidence requirements;
- paired, resumable, multi-fidelity experiments;
- hard constraints and partial-order promotion;
- event-sourced operational custody;
- reindexing across architectures and product adapters.

The rest of the thesis constructs these capabilities from smaller mathematical structures.

2. The empirical anchor: current `ragopt` and `ragkit`

2.1 What already exists

The supplied snapshot contains two valuable but differently scoped packages. `ragkit` has 173 Go files, approximately 17,743 nonblank lines, and 273 test functions across retrieval, chunking, representations, embedding, lexical and exact vector search, reranking, context construction, generation, evaluation, index bundles, execution utilities, flows, and content identity. `ragopt` has 45 Go files, approximately 5,925 nonblank lines, and 42 test functions across candidates, evaluation, comparison, gates, reports, run storage, and command-line adapters. These are static source measurements of the supplied snapshot, not statements about deployment.

`ragopt` already implements several parts of the trusted core that this thesis requires:

- strict, content-identified snapshots;
- a candidate manifest that validates exactly one declared mutation;
- copied and verified assets;
- exact baseline/candidate pairing by case and repeat;
- explicit retention of missing pairs and missing metrics;
- ordered gates and promotion reports;
- durable run directories, append-only records, completion/failure state, and resume checks.

These are not incidental utilities. They are the beginnings of a semantic kernel: identity, custody, pairing, and authorization to promote.

`ragkit` already implements much of the behavioral vocabulary over which an optimization field must range. It has distinct chunking, representation, embedding, index, retrieval, reranking, answering, generation, and evaluation packages. It also contains deterministic ordering and evidence identity rules. The missing piece is not another generic candidate runner. It is a typed account of how changes to these RAG meanings become interventions with closure, preservation claims, and evidence obligations.

2.2 The boundary problem

The generic optimizer should not import every RAG concept. If `ragopt` owns chunk specifications, HNSW parameters, grounding contracts, tool trajectories, and frontend widgets, it becomes a second RAG framework and immediately diverges from `ragkit`. Conversely, a generic optimizer that sees only opaque configuration blobs cannot validate rebuild closure or reuse.

The proposed boundary is an adapter. `ragopt` owns generic intervention custody, statistical experiment execution, decision protocols, and campaign state. A `ragopt/ragspace` package imports stable RAG semantic types from `ragkit` and supplies:

- typed parameter references;
- effect grades;
- dependency graphs and supports;
- observation families;
- fidelity definitions;
- product-neutral RAG evidence obligations.

Applications supply native objectives, suites, traces, and product gates. This direction preserves dependency hygiene:

$$\text{application} \to \text{ragspace} \to (\text{ragkit}, \text{ragopt}),$$

not `ragopt` $\to$ product code.

2.3 Exactly one mutation as a generator

The current `ragopt` one-mutation invariant is useful and should not be discarded. It should be reinterpreted. An atomic candidate is a generator in an intervention category. A compound candidate is a verified path of generators:

$$i_n \circ \cdots \circ i_2 \circ i_1.$$

Each adjacent snapshot identity must agree, and the path identity must be preserved even when two paths produce the same endpoint. This permits factorial or coordinated experiments without returning to opaque multi-change files. It also means the existing candidate format can remain stable while a versioned path manifest is added above it.

3. The semantic object: an evolving RAG service

3.1 Beyond retrieval as a pure function

A minimal retrieval function has type

$$q : Q \to List(E).$$

A production RAG service has source state, release state, caches, provider state, conversation state, time, deadlines, random choices, and streaming output. A more faithful one-step type is

$$T_\theta : S \otimes X \to D(S \otimes Y \otimes Tr),$$

where S is runtime state, X is an input event, Y is an external outcome, Tr is an intensional trace, and D is a probability construction. The parameter θ is behavior-complete: it binds the source barrier, derived artifacts, query policy, provider identities, validators, operational policy, and projection policy capable of changing observable behavior.

The trace is not debugging exhaust. It carries facts needed to distinguish releases that return the same answer text: evidence lineage, disclosure, fallback path, release lease, latency, cost, tool actions, partial delivery, and cancellation. Optimization can project this trace differently for different claims.

3.2 Time and state

Index optimization and query optimization meet at release activation. Source revisions are transformed into immutable releases; queries acquire a lease on one active release; experiments compare release-pinned interpretations. The semantic state therefore contains at least:

$$S = S_{\text{source}} \times S_{\text{build}} \times S_{\text{release}} \times S_{\text{query}} \times S_{\text{campaign}}.$$

The optimizer is not outside this runtime. It observes traces, allocates experiments, creates builds, and proposes promotions. Its own state and transition laws must be modeled. A promotion decision made from incomplete pairing is a runtime safety error even if the comparison code is numerically correct.

3.3 Open-system interfaces

RAG components are open systems: a retriever receives a query and index interface; a reranker receives candidates and possibly calls a provider; a context builder receives an evidence budget; an evaluator receives a case, release, and randomization context. The architecture must preserve these interfaces so that components can be composed and replaced without flattening their semantics.

Wiring-diagram and operadic accounts of dynamical systems motivate this treatment: modular dynamical systems can be assembled by explicit interface wiring rather than by erasing structure into one global transition function [Libkind et al. 2022]. The implementation in this volume uses ordinary Go composition rather than a generic operad library, but the mathematical interpretation treats architecture as wiring data.

4. Three meanings that must agree

4.1 Denotational meaning

The denotation of a behavior-complete release is the stochastic behavior it induces. For a fixed architecture A and specification $\theta \in \Theta_A$:

$$[\![\theta]\!]_A : S \otimes X \to D(S \otimes Y \otimes Tr).$$

For a finite event sequence, Kleisli iteration gives a distribution over final state and trace. Denotation abstracts away from process implementation while retaining explicitly selected intensional events.

4.2 Operational meaning

The operational semantics is a labeled transition relation

$$\langle C, e \rangle \to \langle C', o \rangle,$$

where C is a runtime configuration, e is an input or internal event, and o is an emitted observation. For the optimizer, labels include proposed, built, evaluation-started, evaluation-completed, decided, promoted, and rejected. Illegal transitions are rejected by the reducer.

The operational semantics answers questions denotation alone does not: Can evaluation start before a required build? Can a candidate be promoted without a passing decision? What survives a crash? Which event order is replayable? Which partial state is terminal?

4.3 Statistical meaning

A finite evaluation is neither the denotation nor the operational trace. It is a statistical experiment generated from them. For a suite case c, randomization context z, fidelity f, and release θ:

$$E_{\theta,f}(c, z) \in D(O_f).$$

A comparison observes paired samples and computes interval-valued evidence. Promotion is therefore a decision under uncertainty, conditional on suite scope and modeling assumptions.

4.4 Adequacy obligations

The architecture needs bridges among the three meanings.

1. **Interpreter adequacy:** the distribution of completed operational traces projects to the denotational channel.
2. **Replay adequacy:** folding the append-only event log yields the same campaign state as online reduction.
3. **Experiment fidelity:** an evaluation cell records the release, suite, case, repeat, seed design, policy, and fidelity whose channel it sampled.
4. **Decision adequacy:** the gate consumes only evidence certified for the intervention's effect grade.

These are obligations, not automatic consequences of using category-theoretic language.

5. Design criteria and non-goals

The mathematical core is judged by a practical standard: does it make invalid production actions unrepresentable or rejectable while allowing domain-specific extension?

A successful design should be **compositional**: the meaning and support of a compound intervention should be derivable from its parts. It should be **conservative**: composition must not silently lower effect grade or evidence requirements. It should be **intensional when necessary**: path, disclosure, and release lineage should survive when endpoint output is insufficient. It should be **extensional when justified**: unaffected artifacts and observations should be reusable under a checkable theorem. It should be **statistically honest**: missing pairs, uncertainty, and fidelity limitations should remain visible. It should be **operationally closed**: promotion authority must be implemented by a small reducer rather than convention.

The thesis does not attempt to solve all search algorithms, prove semantic properties of arbitrary external model providers, define a universal metric of answer quality, or certify production security from offline tests. It does not require that every component be differentiable in the ordinary real-valued sense. Its change calculus is finite and structural. It does not claim that every low-fidelity suite is a Blackwell garbling of a high-fidelity suite; instead it makes that relation an explicit claim requiring a witness or calibration.

Part II. Compositional behavioral semantics

6. The base category of stochastic behavior

6.1 Markov kernels

A RAG release contains deterministic code and stochastic components: approximate candidate selection, provider sampling, load-dependent scheduling, retries, timeouts, and external state. The natural base is therefore a category of stochastic maps rather than ordinary functions.

In the finite model, an object is a finite set and a morphism $k : X \to Y$ assigns a probability distribution $k(x)$ over Y to every $x \in X$. Composition is Kleisli composition:

$$(g \circ k)(x)(z) = \sum_{y \in Y} k(x)(y)\, g(y)(z).$$

Identity is the Dirac distribution. Tensor product composes independent channels in parallel. Copy and discard maps on deterministic data give the structure of a Markov category. Markov categories provide an abstract setting for probability, conditional independence, and sufficient statistics across discrete and measure-theoretic models [Fritz 2020].

The sandbox implements this finite fragment in `pkg/kernel`. It checks identity, associativity, tensor interchange, copying, and total-variation equality on finite examples.

6.2 Why copying requires care

A paired experiment should expose both arms to the same case and randomization context. It should not copy a stochastic outcome after sampling it. The categorical distinction is:

$$z \xrightarrow{\Delta} (z, z) \xrightarrow{E_{\theta_0} \otimes E_{\theta_1}} (o_0, o_1),$$

where z is deterministic experimental context containing case, repeat, and seed. Each arm then interprets that shared context. This is the common-random-numbers design implemented by `experiment.SeedFor` and `RunPaired`.

Copying a provider response from one arm into the other would answer a different question. Copying only a nominal seed is also insufficient if provider model, batching, or sampling implementation differs. The experiment identity must say what is actually shared.

6.3 Observation-rich codomains

The semantic output should not be only the user answer. Let

$$Y = O_{\text{answer}} + O_{\text{abstain}} + O_{\text{failure}} + O_{\text{cancel}},$$

and let Tr contain ranked evidence, provider calls, authorization decisions, timing, cost, release, and projection events. Different observation maps select different claims:

$$\pi_{\text{text}} : Y \otimes Tr \to \text{AnswerText}, \quad \pi_{\text{disclosure}} : Y \otimes Tr \to \text{DisclosureTrace}.$$

Two releases may be equivalent under π_{text} and inequivalent under $\pi_{\text{disclosure}}$. Optimization must name the protected observation family rather than asserting unqualified equivalence.

7. Stateful and open RAG transducers

7.1 Stateful Kleisli arrows

For one request, a release is interpreted as

$$T_\theta : S \otimes X \to D\left(S \otimes Y \otimes T r\right).$$

This is a stochastic Mealy-style transducer. Repeated execution is obtained by feeding the next state forward. Conversation turns, cache updates, release leases, and streaming projections are therefore within the semantic object.

The state can be factored by ownership. Source and release state are server-owned. Conversation state may be application-owned. Provider state is external and observed only through an adapter. The factorization matters because an intervention may be local to one state component while preserving others.

7.2 Open composition

Suppose retrieval produces evidence E, context construction produces C, and answer generation produces A:

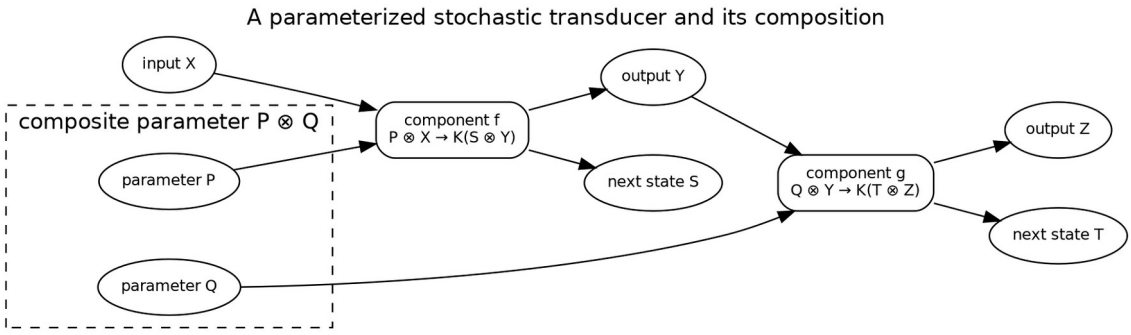
$$R_\rho : S_R \otimes Q \to D\left(S_R \otimes E \otimes T r_R\right),$$

$$K_\kappa : E \to C,$$

$$G_\gamma : S_G \otimes Q \otimes C \to D\left(S_G \otimes A \otimes T r_G\right).$$

Their composite retains separate parameter and trace interfaces. A wiring diagram connects evidence output to context input and context to generation. Parallel structured-fact retrieval can be tensored with text retrieval and then fused. An agent is a feedback system whose next tool action depends on accumulated evidence and model state.

The point of open composition is not diagram aesthetics. It determines where an intervention can focus, which support it has, and which observations can be reused.



A RAG component is a parameterized stochastic transducer; composition accumulates parameters and state.

7.3 Coalgebraic view

A state machine can also be represented coalgebraically as a map

$$c_\theta : S \to \left(D\left(Y \otimes S\right)\right)^X.$$

This view is useful for runtime equivalence and controller design. Bisimulation can express whether two implementations match step by step under selected observations. Trace equivalence is weaker and may suffice for an optimization claim. The thesis does not impose one universal equivalence; it supplies observation-indexed relations in Chapter 14.

8. Parameterized morphisms and the Para construction

8.1 Components with parameters

A component whose behavior depends on a parameter object P is a morphism

$$f : P \otimes X \to Y .$$

The Para construction turns such maps into arrows from X to Y while retaining P as part of the arrow. If $(P, f) : X \to Y$ and $(Q, g) : Y \to Z$, their composite has parameter $P \otimes Q$:

$$(Q, g) \circ (P, f) = \left(P \otimes Q, P \otimes Q \otimes X \xrightarrow{\cong} Q \otimes (P \otimes X) \xrightarrow{1 \otimes f} Q \otimes Y \xrightarrow{g} Z \right).$$

This construction has been used to formalize parameterized learning systems and their composition [Cruttwell et al. 2022]. For RAG, the parameters are not limited to differentiable weights. They include source policies, chunkers, embedding identities, index settings, query plans, timeouts, prompts, and projection policies.

8.2 Behavior-complete parameter objects

Let the architecture contain components $c \in C_A$ with local parameter objects P_c. The global parameter object is a structured product or tensor

$$P_A = \bigotimes_{c \in C_A} P_c .$$

A release specification is a point $\theta : I \to P_A$, where I is the tensor unit, plus content-addressed references to large assets. “Behavior-complete” means that every input capable of changing protected behavior is either inside P_A or explicitly modeled as environmental state. Hidden process environment, mutable provider aliases, or unversioned prompts violate this condition.

This is an engineering discipline rather than a purely mathematical fact. The denotation can only be stable relative to the declared boundary.

8.3 Parameters versus campaign state

Search-policy state is not a release parameter. Momentum, surrogate models, trial budgets, and prior observations belong to campaign controller state. A candidate proposes a new release parameter. Keeping these separate prevents accidental promotion of optimizer internals and makes replay possible.

9. Optics and local intervention

9.1 Lenses as typed focus

A local change needs a way to focus a component parameter inside the whole specification. A cartesian lens from whole S to part A consists of:

$$get : S \to A\,,\ put : S \times A \to S\,.$$

The usual laws are:

$$put\left(s\,, get\left(s\right)\right) = s\,,$$

$$get\left(put\left(s\,, a\right)\right) = a\,,$$

$$put\left(put\left(s\,, a_1\right), a_2\right) = put\left(s\,, a_2\right).$$

These laws state that focusing and rebuilding the release does not create unrelated drift. Lenses and more general optics provide a uniform account of bidirectional accessors and lawfulness [Riley 2018].

The sandbox implements cartesian lenses in `pkg/optic`, composition of lenses, finite law checks, and commuting-update checks.

9.2 Intervention as focused action

Let $\ell_c : P_A \leftrightarrow P_c$ focus component c. Let $\delta_c \in \Delta P_c$ be a local change acting through $\oplus_c$. The induced global intervention is

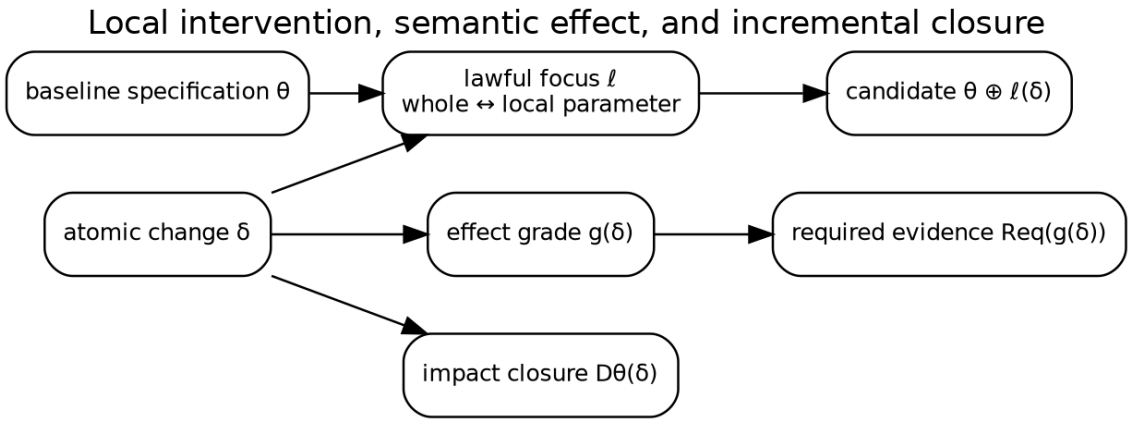
$$i_{c,\delta}(\theta) = put_{\ell_c}\left(\theta\,, get_{\ell_c}(\theta) \oplus_c \delta_c\right).$$

The intervention carries more than this state transformation. It carries a typed target, hypothesis, effect grade, atomic identity, and claimed preservation relation.

9.3 Sequential and parallel composition

Sequential composition is ordinary function or categorical composition. Its target support is the union of constituent supports, and its effect grade is their join.

Parallel composition is valid only for interventions on certified independent or commuting foci. Disjoint field names are not by themselves a proof: two fields can influence the same derived artifact or invariant. A parallel combinator therefore requires both disjoint target support and a domain certificate. The sandbox requires an explicit certificate function and rejects overlap.



A local optic changes one focused parameter while effect grading and dependency differentiation propagate its consequences.

10. From lenses to general optics

Cartesian lenses are sufficient for ordinary immutable Go structs, but the optimization architecture benefits from the broader optic pattern. An optic separates a forward map from the residual context needed for a backward update. In an optimizer, forward behavior produces observations; backward information carries objectives, counterfactuals, or update demands to the focused component.

Categorical cybernetics combines parametrization and optics to describe systems that interact with an environment and controller, with objectives flowing backward [Capucci et al. 2022]. This suggests a long-term formulation in which a RAG component is a parameterized optic and a search controller supplies update policies. The present implementation stops short of a generic optic coend because that machinery would obscure the trusted kernel in Go. It uses lenses for configuration focus, explicit evaluator outputs for feedback, and an event-sourced controller.

This distinction is important. The thesis uses optics as a semantic organizing principle, not as a requirement that application developers manipulate advanced categorical encodings. A package API should expose simple typed interfaces whose laws correspond to the optic interpretation.

11. Change actions and finite derivatives

11.1 Changes as first-class values

Ordinary differentiation models infinitesimal changes in smooth spaces. RAG optimization contains discrete changes: replace a prompt, add a source, switch an index backend, change a chunk boundary, or alter a timeout. Change actions generalize differentiation by equipping a value space A with a monoid of changes ΔA acting on it:

$$\oplus : A \times \Delta A \to A.$$

A derivative of $f : A \to B$ is a function

$$Df : A \times \Delta A \to \Delta B$$

satisfying the fundamental update law

$$f(a \oplus \delta a) = f(a) \oplus Df(a, \delta a).$$

Change actions connect incremental computation and discrete derivatives [Alvarez-Picallo 2020]. The sandbox implements the law directly and checks finite examples.

11.2 Derivatives of build functions

Let

$$B : \Theta_A \to A_A$$

map a release specification to its derived artifact family. A derivative

$$DB(\theta, \delta\theta)$$

is an incremental artifact update whose application agrees with a clean rebuild. This is the formal version of the production oracle:

$$B(\theta \oplus \delta\theta) \simeq B(\theta) \oplus DB(\theta, \delta\theta).$$

For deterministic exact artifacts, $\simeq$ can be byte or semantic equality. For approximate indexes, it is a declared tolerance-relative observation relation.

11.3 Support abstraction

A full derivative may be expensive or backend-specific. A dependency DAG is a conservative abstraction of its support. Each node represents a parameter, artifact, runtime stage, or evaluator. An edge $a \to b$ means that b may semantically depend on a. For changed nodes C, the impact is the least downstream-closed set:

$$cl(C) = \mu X . C \cup \succ(X).$$

This closure need not compute the delta itself. It determines what cannot be reused without a stronger semantic certificate.

12. Dependency closure as an abstract derivative

12.1 Soundness condition

Let every artifact or evaluator v have a denotation F_v whose explicit parent inputs are $pred(v)$. A dependency graph is sound when, for any two worlds agreeing on all ancestors of v, F_v agrees under its declared observation relation. If a changed node cannot reach v, the worlds agree on the support of F_v, so v is reusable.

> **Proposition 12.1 — Support-disjoint reuse.** Let i be an intervention with impact closure I. Let an artifact or evaluator result a factor through a projection onto support S. If $I \cap S = \varnothing$ and all external identities used by a agree, then a has the same denotation before and after i.
>
> **Proof sketch.** The intervention changes only coordinates in I. Disjointness implies the projection onto S is unchanged. Since a factors through that projection, extensionality gives equal output. External identities are separate coordinates and must also agree. ▫

The external-identity clause includes source barrier, release epoch, case suite, randomization design, evaluator policy, provider identity, and fidelity. Content-addressing only the artifact bytes is therefore insufficient.

12.2 Closure algebra

Closure is extensive, monotone, and idempotent:

$$C \subseteq cl(C),$$

$$C \subseteq D \Rightarrow cl(C) \subseteq cl(D),$$

$$cl(cl(C)) = cl(C).$$

It also preserves finite unions in a directed reachability graph:

$$cl(C \cup D) = cl(C) \cup cl(D).$$

These laws make compound-intervention planning compositional. The sandbox checks closure behavior and renders the RAG dependency graph as Graphviz.

12.3 Precision ladder

Dependency analysis can be introduced in levels.

1. **Coarse DAG:** conservative reachability.
2. **Keyed nodes:** reuse when canonical input identities match.
3. **Change-sensitive derivative:** produce an exact affected-key set.
4. **Semantic delta:** incrementally update the artifact and verify against a rebuild oracle.
5. **Proof-carrying reuse:** attach a machine-checkable factorization or refinement witness.

A production migration should begin with conservative closure and make precision improvements only where their cost is justified.

13. Behavior identity and causal identity

A release identity answers, “What behavior-complete parameter point is this?” A candidate identity answers, “By what declared intervention did we move from the baseline to this point?” A build identity answers, “Which subset of parameters determines these artifact bytes?” An evaluation identity answers, “Which experiment was sampled?” These identities must not be collapsed.

Let

$$id_{\text{release}} = H(\theta),$$

$$id_{\text{build}} = H(\pi_B(\theta)),$$

$$id_{\text{candidate}} = H(id_{\text{parent}}, i, id_{\text{child}}),$$

$$id_{\text{experiment}} = H(id_{\text{candidate}}, \text{suite}, \text{policy}, \text{fidelity}, \text{barrier}, \text{randomization}).$$

The sandbox demonstrates the distinction: fusion and reranking changes preserve the build key while changing full spec identity. A chunk-size change changes both.

Path identity also guards causal interpretation. If a search system proposes a compound endpoint, the optimizer can require an ordered sequence of atomic candidates. The final release may be deployable regardless of path, but the optimization record should not pretend that an interaction was an isolated one-factor result.

14. Observation-indexed equivalence and refinement

14.1 No unqualified semantic preservation

An intervention is not simply "semantics-preserving." It preserves a selected observation family O. For each observation map $O \in O$, exact preservation requires

$$O \circ [\![\theta_0]\!] = O \circ [\![\theta_1]\!].$$

Examples include answer text, ranked evidence, authorization decisions, release lineage, frontend projection, and latency-excluded functional outcomes. An operational tuning may preserve successful answers but change deadline-triggered failures; then it is not equivalent under the outcome observation.

14.2 Approximate refinement

Approximate retrieval needs a metric or divergence. Let d_O compare distributions under observation O. An error-budget claim is

$$d_O\left(O \circ [\![\theta_0]\!], O \circ [\![\theta_1]\!]\right) \le \varepsilon_O.$$

The relation should be compositional. If each component has an error budget and the chosen metric admits a composition bound, the whole pipeline can accumulate a conservative budget. In practice, ANN recall relative to exact retrieval is only one coordinate; downstream reranking and context admission may amplify or mask retrieval error.

14.3 Refinement rather than equality

Security and abstention claims are often preorders. A candidate may reveal no more information than the baseline, or abstain on at least the unsafe cases. Write

$$\theta_1 \sqsubseteq_{\text{disclosure}} \theta_0$$

for no-more-disclosing behavior. The direction of "better" depends on the observation. The decision doctrine therefore carries metric directions and predicates explicitly.

14.4 Trace congruence

For compositional reasoning, an observation relation should be a congruence under the wiring contexts in which replacement is allowed. If $f \sim g$ but composing either with a deadline wrapper yields observably different failure behavior, then $\sim$ was too weak for that context. The architecture should attach preservation claims to both an observation family and an admissible context class.

15. The optimization field

15.1 Fiber categories

Fix an architecture A. Define a category $Opt(A)$:

- objects are valid behavior-complete specifications $\theta \in \Theta_A$;
- morphisms $i : \theta \to \theta'$ are validated intervention paths;
- identity is the empty intervention;
- composition concatenates paths and validates adjacent identities.

The slice category

$$(\theta_0 \downarrow Opt(A))$$

is the candidate category for baseline θ_0. An object is an intervention out of the baseline; a morphism between candidates is a further intervention making the triangle commute. This formalizes the statement that candidates are arrows, not points.

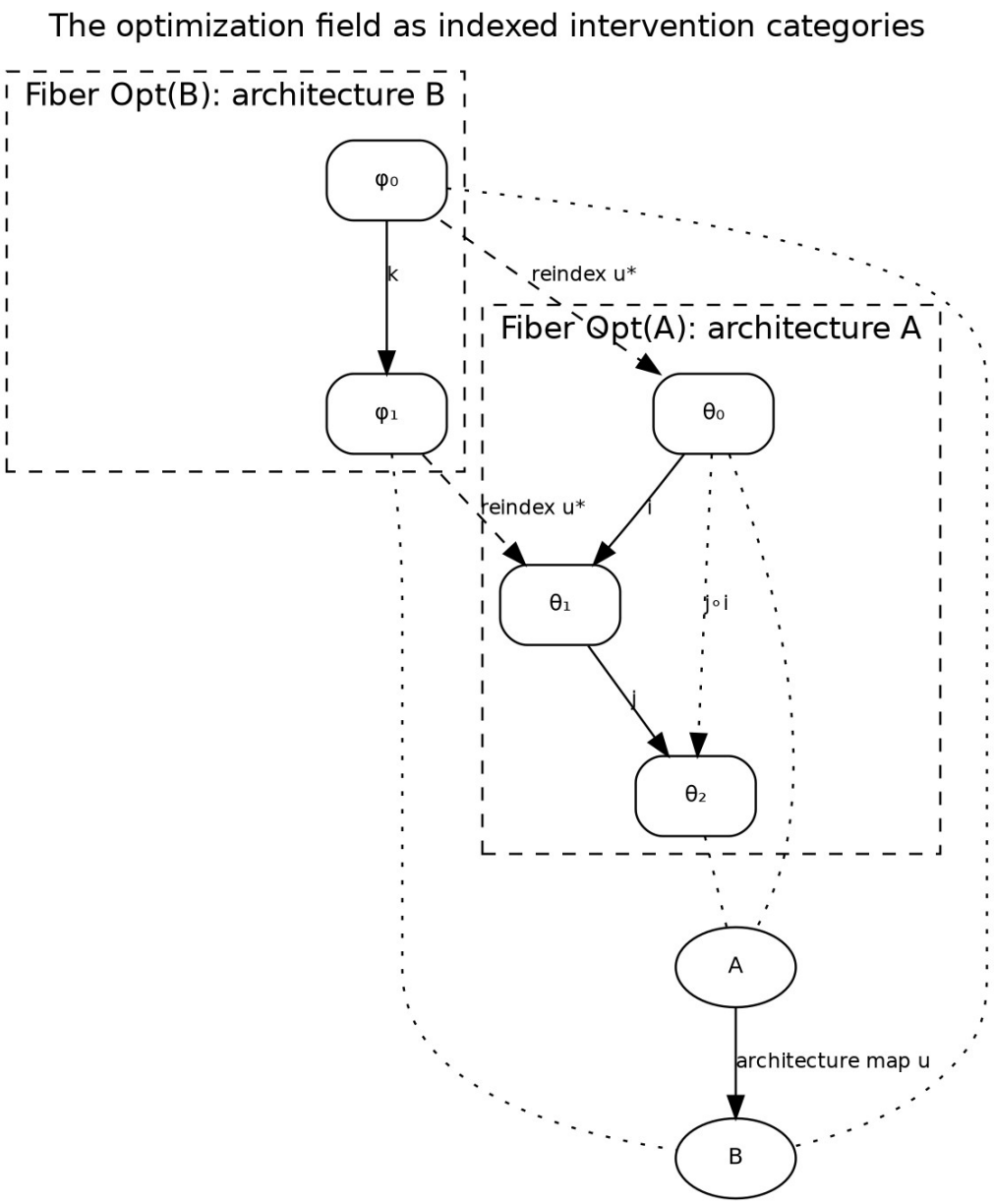
15.2 Architecture indexing

Let A contain architectures and valid architecture mappings. A mapping $u : A \to B$ can induce a reindexing functor

$$u^{\square} : Opt(B) \to Opt(A),$$

for example by restricting a shared RAG specification to a product facade or translating a generic query plan into an application-supported subset. Reindexing must preserve identity and composition up to the coherence chosen by the implementation.

The sandbox's `field.Reindex` is only an executable shadow: it maps specifications and supports and checks identity/composition on finite examples. A production implementation can remain concrete while retaining the law as an API contract.



Each architecture has its own intervention category; reindexing transports specifications and supports between fibers.

15.3 Graded symmetric monoidal structure

Within a fiber, independent components can be composed in parallel. This gives a partial or certified monoidal structure. Interventions are graded by semantic effect in a join-semilattice E:

$$g(1_\theta) = \bot,$$
$$g(j \circ i) = g(i) \vee g(j),$$
$$g(i \otimes j) = g(i) \vee g(j).$$

The grade is a conservative summary, not a complete semantics. It exists to prevent composition from reducing required scrutiny.

15.4 The complete field structure

An optimization field consists of:

$$F = (A, Opt, [\![-]\!], \Delta, D, g, Req, E, D),$$

where:

- A is the architecture category;
- Opt is the indexed intervention category;
- $[\![-]\!]$ interprets specifications as open stochastic systems;
- Δ and D are change actions and derivatives/support closures;
- g is the effect grading;
- Req maps effects to evidence obligations;
- E supplies experiment semantics;
- D supplies decision and campaign semantics.

This tuple is the central abstraction of the thesis. Each later part develops one component and its laws.

Part III. Effects, experiments, and decisions

16. Semantic effect grading

16.1 Why effect is not a tag

An effect grade is an upper bound on the kinds of protected behavior an intervention may change. It drives evidence requirements, invalidation policy, reviewer authority, and deployment controls. A free-form label is insufficient because composition must be lawful.

Let E be a finite join-semilattice with bottom $\bot$. The sandbox uses seven generators:

- **operational** — scheduling, batching, caching, concurrency, or resource policy;
- **approximation** — replacement of an exact operation by a bounded approximation;
- **relevance** — retrieval or context ordering without a claimed source-knowledge change;
- **knowledge** — source admission, normalization, chunking, representation, or model changes that alter what can be retrieved;
- **policy/security** — authorization, redaction, disclosure, retention, or evidence-kind changes;
- **interaction** — answer, agent, session, tool, or conversation behavior;
- **presentation** — frontend projection, choices, widgets, or evidence display.

The elements of E are finite joins of these generators. This is not a claim that the concerns are ontologically independent. It is a conservative bookkeeping algebra.

16.2 Grading laws

For an intervention category I, grading is a map

$$g : Mor(I) \to E$$

satisfying identity and composition laws. A useful implementation strengthens equality to conservatism:

$$g(j \circ i) \geq g(i) \vee g(j).$$

This permits an adapter to escalate a compound intervention when interaction creates a new concern. For example, two individually relevance-changing operations may jointly change interaction behavior because one alters the context available to an agent loop.

The grade must be computed from typed parameter ownership and semantic declarations, not supplied solely by an untrusted search policy. Search may propose a grade; the adapter validates or raises it.

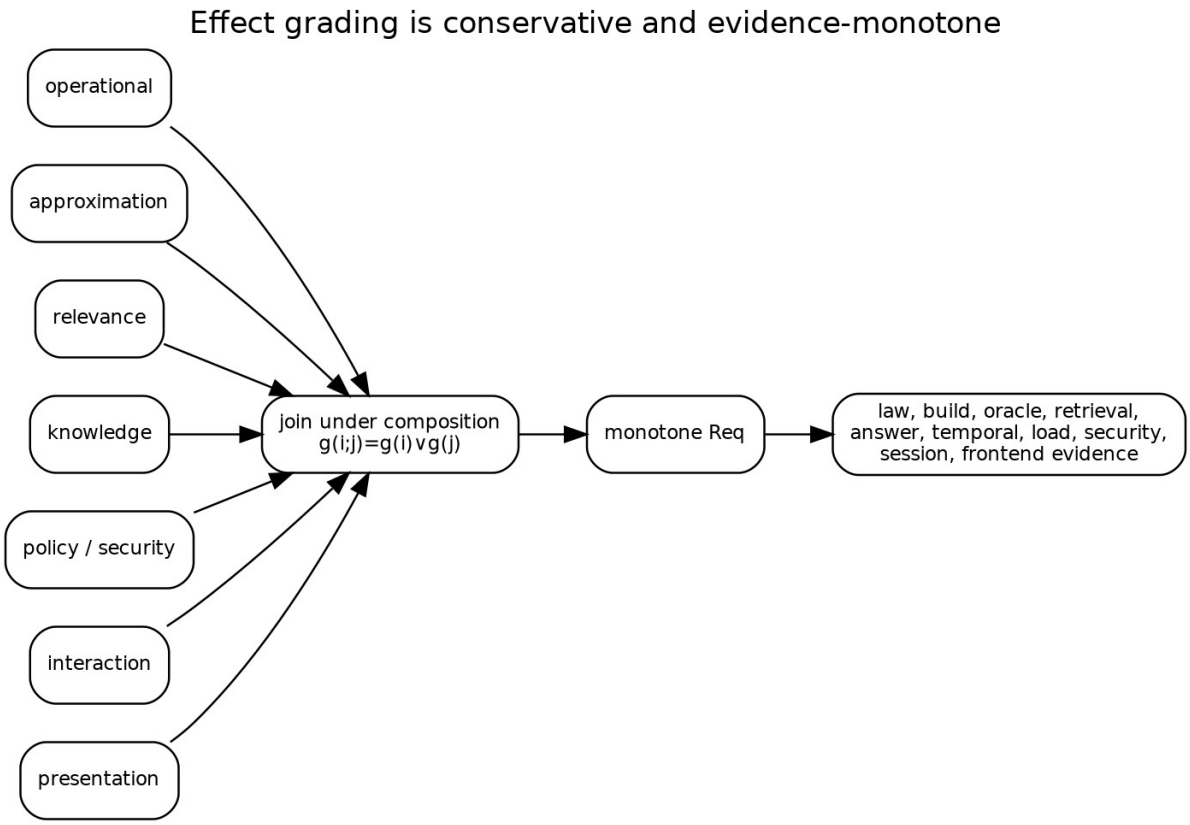
16.3 Grade soundness

> **Definition 16.1 — Grade soundness.** A grading is sound relative to observation families $\{O_e\}_{e \in E}$ when every observation that can differ between θ and $i(\theta)$ belongs to an observation family covered by $g(i)$.

Exact grade soundness is difficult for arbitrary code. Production practice can use a conservative ownership rule: every parameter type declares a minimum grade, and cross-component dependency rules may add grades. Static analysis, tests, and reviewer certificates can refine but not lower the minimum automatically.

16.4 The strictest concern wins

Because E is ordered by containment, a compound candidate cannot escape scrutiny by being described as primarily operational or primarily relevant. If it changes authorization and fusion weights, its grade includes policy/security and relevance. The required evidence is the join of both obligations.



Effect grades join under composition, and the evidence map is monotone.

17. Evidence obligations as a doctrine

17.1 Obligation lattice

Let O be a join-semilattice of evidence obligations. The sandbox includes:

$$\begin{aligned}&\text{static laws},\text{build integrity},\text{exact oracle},\text{retrieval evaluation},\\&\text{answer evaluation},\text{temporal evaluation},\text{load evaluation},\text{security review},\\&\text{session evaluation},\text{frontend evaluation}.\end{aligned}$$

A monotone map

$$Req : E \to O$$

assigns the minimum required evidence. Ideally it preserves joins:

$$Req(e_1 \vee e_2) = Req(e_1) \vee Req(e_2).$$

The sandbox checks monotonicity over the finite basis. A production adapter may add product-specific obligations, so equality can be replaced by a conservative inequality.

17.2 Evidence as a certificate, not a Boolean

Implementation convenience may represent completed obligations as a bitset, but production evidence should be a family of references:

```
type EvidenceCertificate struct {
    Obligation   ObligationID
    Artifact     artifact.Ref
    Subject      CandidateID
    Suite        SuiteID
    Policy       PolicyID
    Fidelity     FidelityID
    Issuer       ActorID
    CompletedAt  time.Time
    ExpiresAt    *time.Time
}
```

The reducer admits a promotion only when valid certificates cover $Req(g(i))$. Some obligations expire when a source barrier, provider, threat model, or deployment topology changes.

17.3 Evidence transport

Reindexing between architectures can transport evidence only if the observation and support maps are preserved. A generic retrieval law test may transport from `ragkit` to every product. A GEC authorization review does not transport to Garden merely because both use the same retriever. This is another use for indexed structure: obligations and certificates live over an architecture and can be pulled back only along validated maps.

17.4 Proof-carrying candidates

The long-term form of a candidate is proof-carrying. It contains machine-checkable witnesses for simple obligations—identity, closure, exact pairing, build verification—and references to empirical or human certificates for the rest. The gate kernel checks the package. This does not eliminate judgment; it makes the boundary between automated proof and review explicit.

18. Statistical experiments as semantic channels

18.1 Experiment object

Fix an architecture, baseline, candidate, suite, fidelity, and experimental policy. Each release induces a channel from latent case world and randomization context to observations:

$$E_{\theta,f} : C \otimes Z_f \to D(O_f).$$

The suite is a finite design over C. The fidelity determines case subset, repeats, provider mode, index scale, trace detail, and possibly observation projection. An experiment artifact records the channel identity as far as the system can control it.

The experiment is not merely a map from case to metric vector. Native outcomes and traces should be retained. Metric extraction is a later observation map. This permits future metrics, auditing, and detection of summaries that were insufficient for a gate.

18.2 Paired design

For baseline θ_0 and candidate θ_1, paired evaluation samples the same cell key

$$k = (\text{case}, \text{repeat}, \text{barrier}, \text{seed context}, \text{cohort})$$

for both arms. A pair exists only when both cells agree on every locked identity. Missing cells remain explicit.

The paired contrast for metric m is

$$d_k = m(o_{1,k}) - m(o_{0,k}).$$

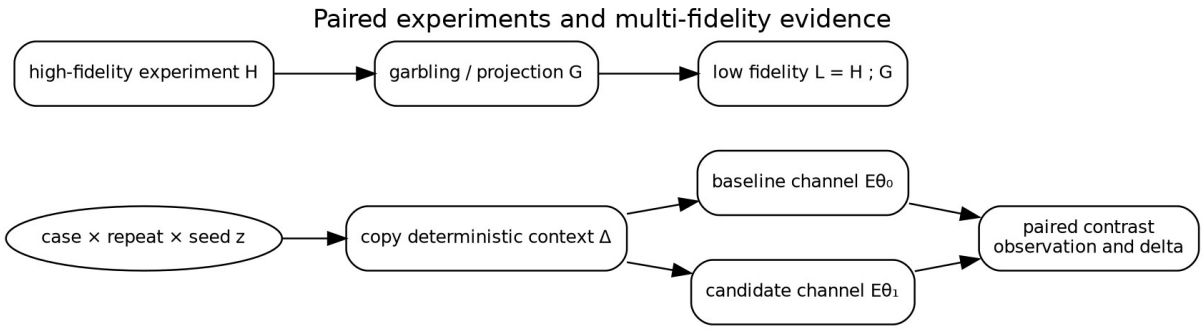
Using $\{d_k\}$ rather than two unrelated sample means removes variation shared by the arms. This is especially important for stochastic generation, approximate retrieval, and provider latency. It does not remove systematic bias from the suite or evaluator.

18.3 Randomness as typed context

A raw integer seed is not a complete randomization design. The manifest should bind:

- seed-derivation algorithm and version;
- provider/model sampling API and parameters;
- whether provider determinism is claimed;
- request grouping and batching policy;
- approximate-index randomization;
- load-generation schedule;
- retry and timeout policy.

Two arms receive the same typed context where meaningful. An arm may legitimately ignore a field. The comparison must not claim common randomness for unpinned external sources.



A paired contrast copies deterministic context into both arms; a valid lower fidelity may factor through a higher experiment by garbling.

19. Multi-fidelity evaluation and Blackwell order

19.1 Fidelity is informativeness, not just cost

A fidelity is often described by case count or expense. Semantically, it is an experiment with a particular observation channel. A high fidelity H is at least as informative as low fidelity L in the Blackwell sense when there exists a garbling channel G such that

$$L = G \circ H .$$

Every decision rule available after observing L can then be reproduced from H. Comparison of statistical experiments and the Blackwell–Sherman–Stein theorem can be formulated in Markov categories [Fritz et al. 2023].

The sandbox provides a finite exact checker for a supplied garbling witness.

19.2 Valid examples

A full trace may be high fidelity and its metric-only projection low fidelity. A complete case suite may be high fidelity and a deterministic case-subset projection low fidelity for decisions restricted to that subset. A high-resolution latency histogram can be garbled into bucket counts.

These are structural factorization claims. They do not say the low fidelity is adequate for a particular product decision; they say it contains no information unavailable from the high fidelity.

19.3 Invalid assumptions

A small synthetic suite is not automatically a garbling of production traffic. A cheap model judge is not automatically a garbling of a human judgment channel. An approximate small index is not automatically a lower-information form of a full-scale index if it changes failure modes. A shorter agent budget can create qualitatively different trajectories.

When no exact witness exists, the system should record a weaker relationship: calibrated predictor, empirical correlation, or merely budget-allocation heuristic. Promotion evidence must be collected at a fidelity adequate for the effect grade, regardless of how search allocation was performed.

19.4 Fidelity chains and stopping

Let $f_0 \preceq f_1 \preceq \cdots \preceq f_n$ be a claimed informativeness chain. A controller can stop a candidate early for hard failure, futility, or budget. It may not infer a final pass from a lower fidelity unless the decision policy explicitly factors through that fidelity or a valid decision-sufficiency witness exists.

This distinction is implemented in the sandbox: a scalar allocation score chooses which candidates receive more budget, but the final promotion reducer uses only final-fidelity evidence and hard gates.

20. Metrics as monoidal reducers

20.1 Mergeable evidence

Distributed evaluation produces shards. A metric reducer should be a commutative monoid $(M, \oplus, 0)$ so shard order and grouping do not change the result. For univariate observations, the sandbox accumulates

$$(n, \sum x, \sum x^2, \min x, \max x)$$

with componentwise merge. This yields mean, sample variance, and a simple interval.

The monoid law supports retries and parallelism, but only when duplicate cells are prevented or deduplicated by identity. An associative reducer does not make duplicate counting correct.

20.2 Sufficient summaries

A summary $s : O^{\square} \to M$ is sufficient for a decision d when there exists $\hat{d}$ such that

$$d = \hat{d} \circ s.$$

If the gate later needs subgroup failures, disclosure traces, or tail latency not retained by M, the summary was not sufficient. Therefore native artifacts should remain immutable even when common summaries are materialized.

Markov-category treatments of sufficient statistics motivate this factorization view [Fritz 2020]. The architecture uses it operationally: summaries are cached only together with their extractor identity, support, and source experiment.

20.3 Stratification

RAG quality is heterogeneous. A total mean can hide regressions in security-sensitive, semantic, multilingual, or accessibility strata. The reducer key should include case groups, and gates may quantify over required groups:

$$\forall g \in G_{\text{protected}}, \Delta m_g \geq -\varepsilon_g.$$

Composition remains monoidal by using a finite map from group key to reducer state.

20.4 Uncertainty

The sandbox uses a normal 95 percent interval for architectural clarity. That approximation is not universally valid. Production metrics may require bootstrap intervals, paired permutation tests, beta-binomial models, survival analysis, cluster-robust errors, or sequential corrections. The semantic requirement is that the estimate type expose uncertainty and that gate semantics state which bound it uses.

21. Constraints as subobjects and predicates

21.1 Feasible releases

Let M be the metric/evidence object. A hard constraint is a predicate $p : M \to 2$ or, categorically, a subobject of feasible observations. The feasible candidate set is the pullback of all required constraint subobjects.

Examples include:

- complete pairing;
- valid build and release identity;
- zero unauthorized disclosure in a certified suite;
- latency upper bound;
- memory and cost budget;
- minimum freshness or oracle agreement;
- no catastrophic subgroup regression.

A candidate outside the feasible set is not made acceptable by a larger quality score.

21.2 Three-valued decisions

Finite uncertainty motivates three outcomes:

$$Verdict = \{\text{pass}, \text{fail}, \text{need-more-evidence}\}.$$

For an upper-bound constraint $m \le t$ with interval $[L, U)$:

- pass if $U \le t$;
- fail if $L > t$;
- otherwise need more evidence.

This prevents treating inconclusive evidence as success or failure. The sandbox's decision kernel stops at the first non-pass in the ordered gate sequence and retains every check.

21.3 Assume–guarantee contracts

A component intervention may carry an assume–guarantee contract:

$$A_i \Rightarrow G_i.$$

For example, changing worker concurrency preserves outcomes assuming no deadline is reached and provider batching is semantically stable. The optimization campaign must test or establish the assumption in the target deployment context. Composition can combine contracts, but circular assumptions require separate resolution.

This is more precise than labeling the intervention "operational." The grade determines the minimum evidence; the contract states the actual preservation claim.

22. Non-inferiority and partial orders

22.1 Protected metrics

A candidate usually targets one metric while protecting several others. For a maximize metric with allowed degradation ε, conservative non-inferiority using a paired delta interval $[L, U)$ requires

$$L \geq -\varepsilon .$$

For a minimize metric:

$$U \leq \varepsilon .$$

A target improvement of at least δ requires $L \geq \delta$ for maximize or $U \leq -\delta$ for minimize. These relations are not total: candidates can be incomparable or inconclusive.

22.2 Why scalarization is unsafe for promotion

A weighted score

$$J = \sum_i w_i m_i$$

can be useful for search allocation. It is unsafe as the sole promotion relation because weights permit compensation across dimensions that may be hard constraints. The score's meaning also changes when metric scaling or variance changes.

The architecture draws a strict boundary:

- untrusted search policies may scalarize to allocate budget;
- the trusted gate kernel first checks evidence, integrity, hard constraints, and protected non-inferiority;
- only feasible candidates enter a Pareto comparison or product review.

The sandbox follows this rule exactly.

23. Pareto order after feasibility

23.1 Product order

For metric directions d_i, define the product preorder on feasible candidates. Candidate a dominates b when it is no worse on every objective and strictly better on at least one. The Pareto frontier contains non-dominated candidates.

Categorical treatments replace scalar objective functions with objective functors and formulate Pareto fronts over resource categories [Marcolli 2022]. The production interpretation here is simpler: objective projections form a typed vector, directions are explicit, and the frontier is computed only after interval-aware gates.

23.2 Interval-aware frontier

There are several reasonable frontier definitions under uncertainty. The sandbox uses conservative gates, then a point-estimate Pareto frontier for reporting. A stricter production policy could define robust dominance:

$$a \succ_R b$$

only if the worst credible value of a is no worse than the best credible value of b on every coordinate. This may yield a large frontier. Bayesian posterior dominance or probability-of-superiority relations are alternatives, but their assumptions must be explicit.

23.3 Human choice remains

A Pareto frontier does not choose one release. Product owners may prefer lower latency, lower build cost, or higher answer support depending on current constraints. The report should expose the tradeoff and the exact rule used for any automated tie-break. The sandbox recommendation uses the lower confidence bound on MRR improvement, then answer support, latency, and stable identity; that rule is demonstrative, not universal.

24. A sound promotion rule

Let $i : \theta_0 \to \theta_1$ be a candidate, $g(i)$ its effect grade, H its completed evidence certificates, C the hard-constraint predicate, N protected non-inferiority, and T target improvement. The abstract promotion predicate is

$$Promote(i, H) \Leftrightarrow Req(g(i)) \leq H \wedge C(i, H) \wedge N(i, H) \wedge T(i, H).$$

Proposition 24.1 — Monotone scrutiny. If $g(i) \leq g(j)$ and j is admissible with evidence H, then H covers the minimum obligations of i.

Proof. Monotonicity of Req gives $Req(g(i)) \leq Req(g(j)) \leq H$. ▫

Proposition 24.2 — No scalar bypass. If the reducer implements the conjunction above and search proposals cannot write promotion events directly, no allocation score can promote a candidate that fails a hard predicate.

Proof sketch. The only transition to promoted state is guarded by a passing decision. The decision reducer constructs pass only after each ordered conjunct succeeds. Search scores are absent from that transition rule. ▫

The propositions are modest but operationally important. They identify a small trusted computing base whose implementation can be inspected and model-checked.

Part IV. The runtime dynamics of optimization

25. The optimizer as a cybernetic controller

25.1 Controller, plant, and observation

The object commonly called an optimizer is not the objective function. It is a controller interacting with a plant. The plant is the build-and-serve RAG system; actions include propose, build, evaluate, allocate more fidelity, stop, and request promotion; observations include build outcomes, paired results, costs, failures, and gate decisions.

Let campaign state be C, observations O, and actions A. A possibly stochastic controller has type

$$\Gamma : C \otimes O \to D(C \otimes A).$$

Bayesian optimization, random search, evolutionary search, hand-authored plans, and human proposals are different implementations of Γ. The semantic kernel should not trust any of them with direct promotion authority. It validates every action against the campaign operational semantics.

This separation resembles categorical cybernetics: an open system interacts with both environment and controller, while objective information flows back to parameter updates [Capucci et al. 2022]. RAG optimization extends the pattern beyond gradient updates to discrete, typed interventions and constrained decisions.

25.2 Search policy is replaceable

The controller may maintain surrogate models, acquisition functions, bandit posteriors, interaction estimates, or simple queues. None belongs in candidate identity. Its state can be checkpointed for efficiency, but the append-only campaign log remains the authoritative record of work and decisions.

A failed or upgraded search controller can be replaced and reconstruct its observations from immutable artifacts. A reducer bug is more serious: it can corrupt authorization to build, evaluate, or promote. This asymmetry justifies concentrating invariants in a small package.

25.3 Objectives as feedback

A component objective may be local—retrieval recall, ANN agreement, reranker calibration—or global—answer correctness, task completion, customer outcome. Local objectives are useful for allocation but can be non-congruent with downstream behavior. The optimization field therefore tracks which observation functor produced each objective and which dependency support it covers.

An objective is not assumed to flow backward like an ordinary gradient. It can induce a proposal distribution over discrete interventions, a counterexample, or a request for more evidence. Optic language remains useful because it separates forward behavior from backward control information without imposing smoothness.

26. Small-step operational semantics of campaigns

26.1 Configurations

A campaign configuration is

$$C = (n, C, a),$$

where n is the next event sequence, C maps candidate IDs to candidate state, and a is the active promoted candidate, if any. Candidate state contains:

$$(\text{effect}, \text{requiresBuild}, \text{built}, \text{started}, \text{completed}, \text{decision}, \text{promoted}, \text{rejected}).$$

Events are immutable values. Reduction is a partial function

$$reduce : C \times Event \rightharpoonup C.$$

Undefined cases are illegal transitions and return errors rather than silently mutating state.

26.2 Transition rules

Representative rules are written below. Side conditions are part of the rule.

Propose

$$\frac{c \notin dom(C)}{C \xrightarrow{propose(c,e,b)} C[c \mapsto new(e,b)]}$$

Build

$$\frac{c \in C \; requiresBuild(c) \; \neg terminal(c)}{C \xrightarrow{built(c,h)} C[c.built := \top]}$$

Start evaluation

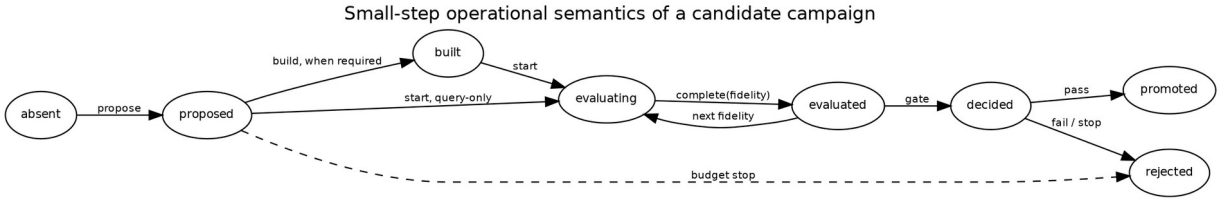
$$\frac{c \in C \; (\neg b_c \vee built(c)) \; f \notin c.started \; \neg terminal(c)}{C \xrightarrow{start(c,f)} C[c.started(f) := \top]}$$

Complete evaluation

$$\frac{f \in c.started \; f \notin c.completed}{C \xrightarrow{complete(c,f,h)} C[c.completed(f) := \top]}$$

Promote

$$\frac{c.decision = \text{pass} \; \neg c.rejected}{C \xrightarrow{promote(c)} C[c.promoted := \top, a := c]}.$$



Candidate work is an explicit transition system; promotion is reachable only through a passing decision.

26.3 Safety invariants

The reducer checks at least:

1. no promoted candidate lacks a passing decision;
2. no candidate is both promoted and rejected;
3. no build-requiring candidate begins evaluation before build completion;
4. no evaluation completes before it starts;
5. the active candidate is promoted;
6. event sequence is contiguous;
7. terminal candidates do not accept new work.

The Go model checker explores all accepted successors for one build-requiring and one query-only candidate to bounded depth. At depth nine it visits 477 unique states and 1,496 legal transitions, checking the invariants in every successor.

26.4 Liveness and fairness

Safety says what never happens; liveness says desirable work eventually progresses. Liveness depends on scheduling assumptions external to the reducer. Candidate work may remain proposed forever if no worker is fair. A production model can add temporal properties:

$$\Box\left(queued\left(c\right) \wedge enabled\left(c\right) \Rightarrow \Diamond\ started\left(c\right)\right),$$

subject to cancellation and budget policy. The included TLA+ model is a starting point for safety; it does not claim a verified fairness model.

27. Event denotation and replay

27.1 Free event monoid

Finite event sequences form a free monoid $Event^{\square}$ under concatenation. The reducer induces a partial action on state:

$$fold : C_0 \times Event^{\square} \rightharpoonup C .$$

Associativity of list concatenation and deterministic reduction give replay composition:

$$fold\left(C_0, u \cdot v\right) = fold\left(fold\left(C_0, u\right), v\right),$$

whenever both sides are defined.

This law permits checkpointing: a snapshot at sequence n plus the suffix denotes the same state as folding the whole log, provided snapshot identity and sequence are verified.

27.2 Operational-denotational correspondence

The denotation of a campaign trace is its folded state plus references to immutable artifacts. The operational interpreter writes events; the denotational fold reconstructs authority. The correspondence obligation is straightforward because both use the same reducer:

> **Proposition 27.1 — Replay adequacy.** If online event append applies `Reduce` before durable publication and replay applies the same deterministic reducer in sequence order, online state equals replayed state after every committed prefix.
>
> **Proof.** Induction on prefix length. The base state agrees. The inductive step applies the same partial function to equal states and the same next event. ▫

Crash safety additionally requires atomic append or a durable log protocol that distinguishes committed records from torn writes. That storage proof lies below the abstract reducer.

27.3 Native artifacts remain outside the state

Large evaluation outcomes, indexes, and reports are content-addressed artifacts referenced by event digests. The reducer state records authority and completion, not mutable embedded copies. This keeps replay bounded and allows independent artifact verification.

28. Search spaces as generated categories

28.1 Atomic generators

A typed search space supplies atomic intervention generators. For a baseline θ, a generator family may be dependent:

$$G(\theta) = \sum_{p : P(\theta)} \Delta_p(\theta).$$

The available changes can depend on current architecture and parameter values. HNSW search breadth exists only when the backend supports it. An overlap value must be less than chunk size. A remote reranker policy is unavailable when disclosure policy forbids it.

The sandbox's `field.Space` provides finite enumeration, product, dependent bind, map, and filter. It is an executable model of dependent candidate construction, not a general dependent type system.

28.2 Paths and free categories

Atomic generators induce a free path category before semantic equations are imposed. Validation quotients or rejects paths according to laws:

- identity changes disappear;
- adjacent replacements of the same lens may collapse for endpoint construction, while audit path remains;
- certified independent changes commute;
- incompatible changes have no composite;
- normalization rules provide canonical search identities where safe.

The distinction between path identity and normalized endpoint identity is intentional. Search may deduplicate builds by endpoint while retaining separate causal histories for analysis.

28.3 Product spaces only after independence

A cartesian product of parameter values assumes they can vary independently. The optimization field instead constructs a tensor/product space only when component foci and validity predicates permit it. Even then, statistical interactions can exist downstream. Algebraic independence of updates means order does not alter the specification; it does not mean their quality effects are additive.

Factorial designs are therefore represented as composed interventions with explicit interaction analysis, not as an unexamined global grid.

29. Commutativity, interference, and interaction

29.1 Update commutativity

For interventions i and j, specification-level commutativity is

$$j(i(\theta)) = i(j(\theta)).$$

Lawful disjoint lenses often imply this equality, but shared invariants or derived normalization can break it. A finite sample check is evidence, not a universal proof. Production code should use typed construction or a domain certificate.

29.2 Behavioral interference

Even commuting updates can interact behaviorally:

$$\Delta_{i,j} m = m(j(i(\theta))) - m(i(\theta)) - m(j(\theta)) + m(\theta).$$

A nonzero interaction term means the joint effect is not additive. Search policy may use factorial or sequential designs to estimate it. Candidate grade and closure remain the joins of constituent concerns; interaction can escalate grade if it reaches new observation families.

29.3 Operational interference

Two candidates may compete for build resources, caches, provider quota, or deployment cohorts. This is campaign-level interference even when release semantics are independent. The scheduler should model resource plans separately from release parameters. Experimental policy may serialize work or randomize execution order to control contamination.

29.4 Concurrency control

The event reducer serializes authority, but workers can run concurrently under leases. A work lease binds candidate, fidelity, attempt, input digests, and deadline. Completion is accepted only if it matches the active lease or an idempotent completion key. This extends the same identity discipline used for source revisions and release activation.

30. Dependency-aware build and evaluation reuse

30.1 Artifact support

An artifact a has support S_a, the set of semantic inputs through which its denotation factors. For a lexical ranking cache this may include normalized query, analyzer identity, lexical index release, filter policy, and depth. For a fused ranking it also includes vector ranking, fusion policy, and representation-collapse rule. For an answer evaluation it includes context, generation provider, prompt, validator, and judge policy.

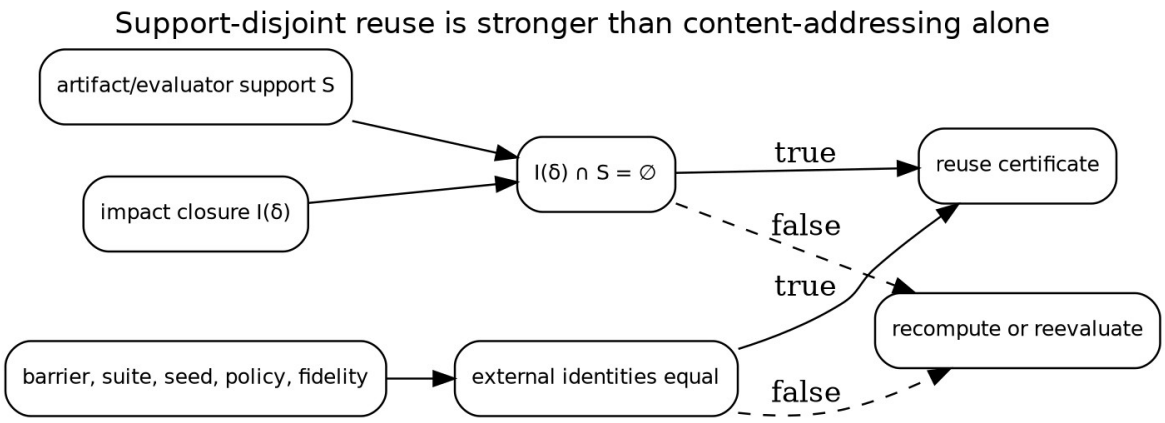
Support should be declared by the owner of the artifact type, not inferred from filenames.

30.2 Reuse predicate

Let I_i be the impact closure of intervention i. Let K_a be external identities. A conservative reuse predicate is

$$Reuse(a, i) \Leftrightarrow I_i \cap S_a = \varnothing \wedge K_a^{\text{before}} = K_a^{\text{after}}.$$

If the intersection is nonempty, reuse may still be possible with a stronger semantic witness. For example, a new reranker pool of four can reuse a stored fused prefix of twenty if the fusion inputs agree. This is represented as a different artifact with support and projection law, not an exception hidden in scheduler code.



Reuse requires both support disjointness and equality of the external experiment identities.

30.3 Build-key projection

A behavior-complete spec θ often has a build projection $\pi_B(\theta)$. Build identity is

$$H_B(\pi_B(\theta)).$$

Query-only changes can reuse the build when this identity agrees and the build support is disjoint. The sandbox's baseline and recommended candidate share a build key even though vector mode, candidate depth, reranking, and context count differ.

30.4 Evaluation reuse

Evaluation reuse is stricter than artifact reuse because a summary may depend on case selection, randomization, judge policy, and grouping. Reusing baseline cells across candidate comparisons is legitimate when the baseline release and experiment cell key agree exactly. Reusing candidate cells across different candidates is legitimate only when the interpreted release and evaluator support agree, not merely when one headline parameter matches.

30.5 Shared intermediate work

A dependency-aware scheduler can construct a work DAG from candidate closures. Candidates with the same chunk/embedding projection share a build. Candidates with identical channel inputs share rankings. Multi-fidelity suites can share native cells when the high-fidelity experiment contains the low-fidelity observation as a projection. The scheduler remains an optimization; correctness is determined by support and identity.

31. Multi-fidelity allocation as a policy over the field

A search campaign selects a finite subcategory or path set from the candidate slice and allocates experiment actions. Its policy can be written as

$$\Gamma_t(H_t, B_t) \in D(Action),$$

where H_t is observed history and B_t remaining budget. Actions include generating a new intervention, evaluating an existing candidate at fidelity f, or stopping.

The field structure improves allocation in several ways.

- Effect grades rule out cheap fidelities that cannot discharge required evidence.
- Closure estimates expose build cost before execution.
- Reuse support lowers marginal cost for candidate families.
- Architecture fibers prevent invalid parameter combinations.
- Path identity supports causal analysis and interaction estimation.
- Blackwell relations identify when higher-fidelity artifacts subsume lower observations.
- Hard-gate failures can stop candidates without estimating a preference score.

A controller may still use Bayesian optimization or other conventional methods inside a finite typed fiber. The thesis does not replace search algorithms; it gives them a semantically valid action space and a trusted boundary.

32. Online optimization and production feedback

32.1 Offline and online experiments are different fibers

Offline evaluation, shadow traffic, canary deployment, interleaving, and A/B tests have different architectures and observation channels. They should be modeled as related fibers rather than as one fidelity integer. Reindexing can transport a release candidate into an online experiment architecture while adding deployment parameters, cohort policy, and privacy obligations.

32.2 Promotion stages

A realistic promotion path may be:

$$\text{static} \to \text{offline} \to \text{shadow} \to \text{canary} \to \text{cohort} \to \text{general}.$$

Each arrow is itself an intervention on deployment state, with operational and possibly policy/security effects. Passing offline gates authorizes entry into shadow evaluation; it does not prove general deployment safety.

32.3 Feedback contamination

Production behavior changes the data later used for optimization. Search results affect clicks, answers affect follow-up queries, and source coverage affects reported failures. The experiment manifest must distinguish logged-policy data from randomized or counterfactual evidence. Continual optimization should not silently train on outcomes generated by an unrecorded prior policy.

32.4 No self-mutating release

The active release should not mutate its own behavior in place based on online feedback. Learning produces a new immutable candidate and follows the campaign transition system. This preserves rollback, pairing, and causal identity. Adaptive per-request behavior can still exist when its policy and state transition are part of the release semantics.

Part V. A composable package architecture

33. Architectural decomposition

33.1 Four ownership layers

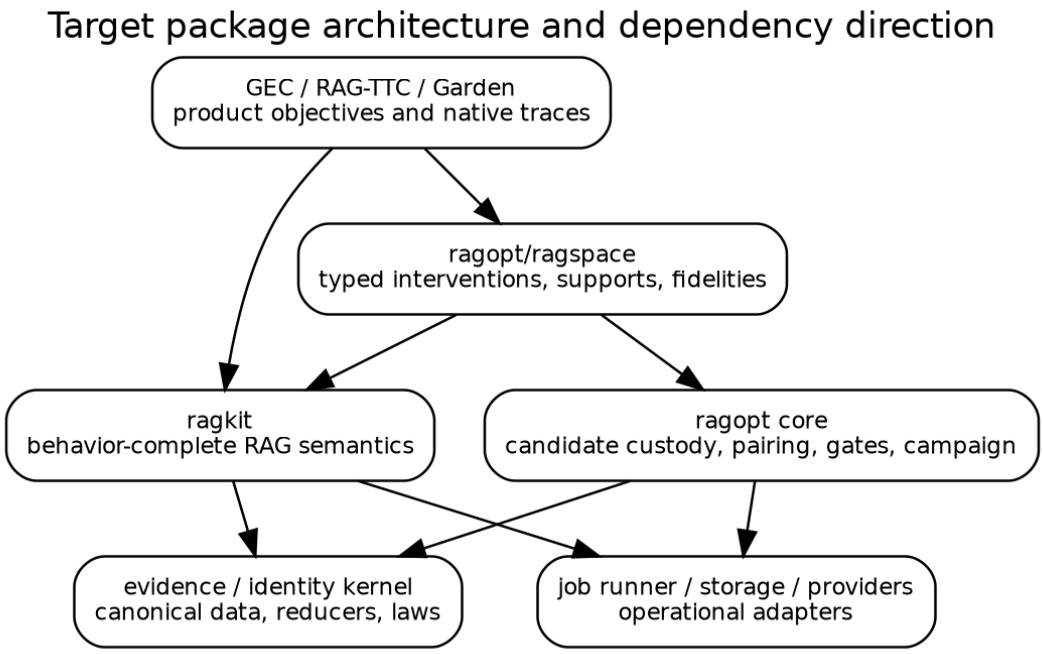
The target architecture has four ownership layers.

Foundational kernel. Canonical identity, immutable references, finite ordering, append-only events, reducers, law-test helpers, and generic algebraic interfaces. This layer contains no RAG vocabulary.

`ragkit`. Behavior-complete RAG meaning: source and release references, chunk and representation specifications, index and retrieval plans, evidence lineage, context and answer contracts, agent/tool semantics, trace types, and RAG dependency support.

`ragopt`. Domain-neutral optimization custody: atomic and path candidates, experiment manifests, exact pairing, run state, mergeable comparison artifacts, ordered gates, campaign reducer, and reports.

RAG adapter and applications. `ragopt/ragspace` maps `ragkit` parameter references to effect grades, closures, observation families, and default obligations. GEC, RAG-TTC, and Garden add product suites, native metrics, security policy, interaction semantics, and presentation outcomes.



The RAG adapter depends on both semantic and optimization cores; neither core depends on applications.

33.2 Dependency rule

A package that owns a semantic type also owns its validation and support declaration. `ragopt` should not switch on string values such as `chunk_size` or `hnsw_ef_search`. `ragspace` should not reimplement retrieval. Applications should not define competing common candidate or pairing formats.

The dependency rule is enforced in source by import-boundary tests. The existing `ragkit` boundary tests provide a model for this practice.

33.3 Why not one package

Combining all concerns into one framework creates three failures. First, optimization mechanics and RAG semantics evolve at different rates. Second, product-specific behavior leaks into shared packages. Third, the trusted promotion kernel becomes too large to audit. The architecture instead composes narrow interfaces and preserves native artifacts by reference.

34. Core Go interfaces

The following APIs are illustrative. They favor explicit values over reflection and stringly typed maps.

34.1 Architecture and specification

```go
type ArchitectureID string

type Specification interface {
    Architecture() ArchitectureID
    BehaviorID() digest.Digest
    Validate(context.Context) error
}

type ParameterRef interface {
    Architecture() ArchitectureID
    StableName() string
    MinimumEffect() Effect
}
```

Large assets remain content-addressed references inside a concrete specification. `BehaviorID` is computed from canonical semantics, not process-local serialization accidents.

34.2 Atomic interventions

```go
type AtomicIntervention[S Specification] interface {
    ID() digest.Digest
    Target() ParameterRef
    Before() artifact.Ref
    After() artifact.Ref
    Hypothesis() string
    Effect() Effect
    Apply(context.Context, S) (S, error)
}
```

The implementation validates that `Effect()` contains `Target().MinimumEffect()`. Atomic before/after references enable the existing `ragopt` one-mutation discipline.

34.3 Verified paths

```go
type Path[S Specification] struct {
    Parent S
    Steps  []AtomicIntervention[S]
    Child  S
    ID     digest.Digest
}

func VerifyPath[S Specification](ctx context.Context, p Path[S]) error
```

Verification applies each step, checks adjacent identities, joins effect grades and target support, validates the child, and computes path identity. Endpoint build deduplication uses `Child`; causal analysis uses `Path.ID`.

34.4 Dependency doctrine

```go
type NodeID string

type DependencyDoctrine[S Specification] interface {
    Graph(S) (DAG, error)
    Changed(Path[S]) Set[NodeID]
    ArtifactSupport(artifact.Kind) Set[NodeID]
    EvaluatorSupport(EvaluatorID) Set[NodeID]
}

type ImpactPlan struct {
```

```
    Changed       Set[NodeID]
    Closure       Set[NodeID]
    Rebuild       []artifact.Kind
    Reevaluate    []EvaluatorID
    Reusable      []artifact.Ref
    Witnesses     []ReuseCertificate
}
```

The plan is deterministic and content-identified. A scheduler consumes it but cannot weaken it.

34.5 Experiment API

```
type CellKey struct {
    Suite      digest.Digest
    Case       string
    Repeat     int
    Barrier    digest.Digest
    Randomizer digest.Digest
    Cohort     string
}

type Fidelity struct {
    ID          digest.Digest
    Rank        int
    Design      artifact.Ref
    Obligations field.Obligation
}

type Cell struct {
    Key          CellKey
    Arm          Arm
    Release      digest.Digest
    Native       artifact.Ref
    Observation  artifact.Ref
    Completed    bool
    Valid        bool
    Failure      *Failure
}
```

Pairing is equality of `CellKey` plus locked experiment identities. Native results are never replaced by a metric-only row.

34.6 Decision API

```
type DecisionPolicy interface {
    ID() digest.Digest
    Required(effect Effect) ObligationSet
    Evaluate(context.Context, CandidateEvidence) Decision
}

type Decision struct {
    Verdict Verdict
    Checks  []Check
    Inputs  []artifact.Ref
    Policy  digest.Digest
}
```

The decision is immutable. Promotion references the passing decision digest. Changing policy requires a new decision artifact even when the candidate observations are reused.

35. `ragkit` optimization semantics

35.1 Behavior-complete release specification

`ragkit` should expose a release specification organized by semantic layer, not one enormous options struct:

```
type ReleaseSpec struct {
    Corpus       corpus.Spec
    Derivation   derive.Spec
    Indexes      index.Spec
    Query        query.Spec
    Ranking      ranking.Spec
    Context      context.Spec
    Answer       answer.Spec
    Agent        agent.Spec
    Serving      serving.Spec
    Presentation presentation.Spec
}
```

Each subpackage owns typed parameter references, validation, build projection, trace projection, and dependency support.

35.2 Parameter ownership

Examples follow.

- `chunking.FixedSizeRef` has minimum effect `knowledge`.
- `vector.SearchBreadthRef` has minimum effects `approximation + relevance + operational` when it affects latency.
- `ranking.FusionWeightRef` has minimum effect `relevance`.
- `serving.WorkerCountRef` has minimum effect `operational`.
- `evidence.AuthorizationOrderRef` has minimum effect `policy/security`.
- `agent.IterationBudgetRef` has minimum effects `interaction + operational`.
- `presentation.WidgetPolicyRef` has minimum effect `presentation + interaction`.

The minimum is conservative. A concrete change can add effects based on value or context.

35.3 Dependency schema

The shared RAG graph should include parameter, artifact, runtime, and evaluator nodes:

```
source/admission
  -> normalized documents
  -> chunks
  -> representations
  -> embeddings
  -> vector index

chunks -> lexical index
query rewrite -> channel queries
indexes + channel queries -> channel rankings
rankings -> fusion -> reranking -> context -> answer/agent -> projection
```

Structured facts, authorization, remote disclosure, release leases, timeouts, and frontend reducers must also appear when they affect behavior. A graph that ends at ranked chunks is not a production RAG dependency doctrine.

35.4 Observation families

`ragkit` should define portable observation types without choosing product metrics:

- retrieval trace;
- evidence lineage and authorization trace;
- context admission trace;
- answer/abstention/failure outcome;
- grounding validation;
- agent/tool trajectory;
- latency/cost/resource trace;
- release and freshness trace;
- frontend projection events.

Applications map these observations to metrics and judgments. This prevents `ragopt` from learning product meaning while enabling shared support and effect rules.

36. `ragopt` evolution

36.1 Preserve the current strengths

The current package should retain strict loading, snapshot digest validation, path-escape prevention, exact mutation declarations, exact pairing, explicit missing data, ordered gates, immutable reports, and durable run custody. These are compatible with the proposed theory.

36.2 Candidate versioning

A migration can proceed without breaking `candidate/v1`.

Version 1: one mutable asset, interpreted as an atomic generator.

Intervention descriptor: a sibling artifact binds that mutation to typed target, effect grade, hypothesis, claimed preservation, and support.

Path version 2: an ordered list of version-1 candidates whose child/parent snapshot identities join exactly. The manifest records path digest and final snapshot.

A compatibility command can derive a conservative descriptor for old candidates, requiring manual confirmation when target semantics are unknown.

36.3 Evaluation versioning

Current exact cells should be extended with:

- source barrier and release ID;
- fidelity ID and design artifact;
- randomization design;
- evaluator ID and support digest;
- native outcome reference;
- observation-extractor identity;
- effect obligations the run claims to discharge.

Comparison remains exact and can continue to reject cross-identity cells.

36.4 Gate versioning

The gate package can add interval-valued predicates without replacing existing deterministic thresholds. A policy is an ordered list of typed checks. Pareto computation belongs in reporting after passing candidates are identified, not inside the authority transition.

36.5 Campaign package

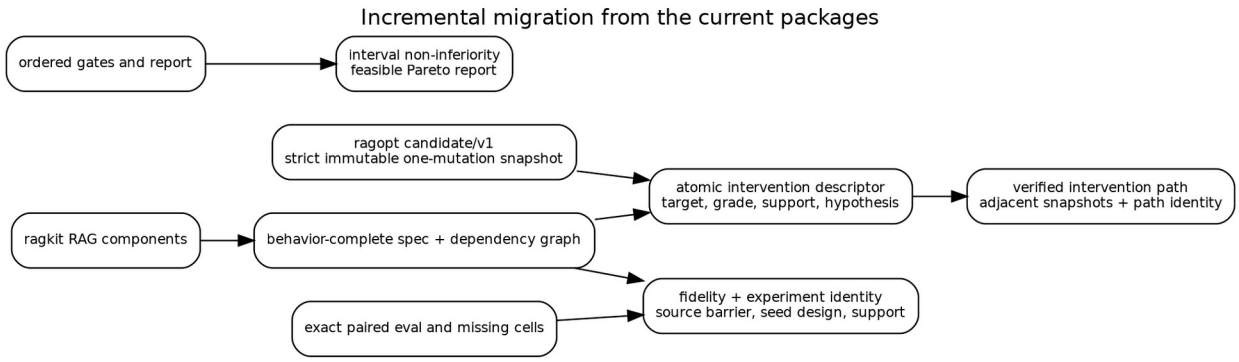
A new small campaign package owns the event types and reducer. Existing runstore directories remain the custody unit for individual builds or evaluations. The campaign log references runs by digest. Search controllers and job workers interact through commands accepted by the reducer.

36.6 RAG adapter

`ragopt/ragspace` should be a thin adapter, not a second implementation of this entire thesis. Its initial responsibilities are:

```
type Adapter interface {
    ValidatePath(context.Context, rag.ReleaseSpec, candidate.Path) error
    Effect(candidate.Path) field.Effect
    Impact(candidate.Path) (rag.ImpactPlan, error)
    Required(field.Effect) field.Obligation
    DefaultFidelities(candidate.Path) []experiment.Fidelity
}
```

Applications can override fidelity and gates upward. They cannot remove obligations required by the shared adapter without a versioned policy exception.



Current immutable candidates, exact pairing, and ordered gates can evolve incrementally into typed paths, experiment identity, and feasible Pareto reporting.

37. Integrating indexing optimization

37.1 Indexing as a parameterized incremental program

The build interpreter is

$$B_\theta : S_{\text{source}} \to D(ReleaseArtifacts \otimes BuildTrace).$$

Most stages are deterministic relative to pinned providers and source barriers, but the stochastic codomain permits remote generation, retries, and approximate construction. The derivative doctrine computes an impact plan for a parameter or corpus change.

Indexing candidate dimensions include source admission, normalization, chunking, representation generation, embedding, lexical analyzers, vector backends, partitions, overlays, compaction, and publication policy. Their effect grades are not identical. Source admission and chunking are knowledge-changing; exact-to-ANN is approximation-changing; worker count is operational unless deadline behavior changes.

37.2 Build integrity obligations

A knowledge-changing candidate generally requires:

- source barrier and input snapshot identity;
- deterministic or observed derivation lineage;
- content-addressed artifact manifest;
- clean rebuild or incremental-equivalence oracle;
- tombstone and deletion tests;
- activation compatibility;
- retrieval and answer evaluation;
- temporal/freshness evaluation when maintenance behavior changes.

The build is not considered complete merely because files exist. Publication is an immutable state transition with verification.

37.3 Rebuild closure examples

Changing chunk overlap reaches chunks, lexical index, representations, embeddings, vector index, rankings, contexts, and answer evaluations. Changing vector search breadth reaches vector ranking, fusion, downstream context, answer, retrieval evaluator, and operations evaluator, but not index bytes when it is query-time only. Changing fusion weight reaches fusion and downstream nodes while channel rankings remain reusable.

The machine-readable graph turns these statements into scheduler inputs and validation rules.

37.4 Joint indexing/query optimization

Indexing and querying cannot be optimized independently in general. Smaller chunks may improve precise retrieval but require larger candidate pools and context diversity. Representation prompts alter embedding neighborhoods and reranker inputs. ANN approximation interacts with fusion and reranking. The field represents these as composed interventions with a joint closure and evaluation path, preserving each atomic causal step.

38. Integrating query and agent optimization

38.1 Query interpretation pipeline

A query release can be decomposed into open components:

$$Q \xrightarrow{\text{rewrite}} Q^{\square} \xrightarrow{\text{channels}} R_1 \otimes \cdots \otimes R_n \xrightarrow{\text{fusion}} R \xrightarrow{\text{rerank}} R' \xrightarrow{\text{context}} C \xrightarrow{\text{answer/agent}} Y.$$

Each component has local parameters, traces, and observation support. The global `Para` composition accumulates their parameter objects; lenses focus changes without flattening the pipeline.

38.2 Approximation interventions

Exact-to-ANN changes should name an exact oracle and tolerance observations. A useful evidence vector includes candidate recall relative to exact, rank overlap, downstream context agreement, answer non-inferiority, latency, tail latency, memory, and failure behavior. Search-depth changes that reduce candidate coverage are approximation and relevance effects, not merely operational tuning.

38.3 Reranking and remote disclosure

A reranker intervention may affect relevance, latency, cost, provider failure, and security. If hydrated source text crosses a remote boundary, authorization order and disclosure certificates are in support. An optimizer cannot approve a relevance gain that violates the policy/security subobject.

38.4 Context and agent behavior

Context count, token budget, diversity, and ordering affect answer support and interaction. Agent tool descriptions, iteration budgets, and repair policies change trajectory distributions. Evaluation must retain native trajectories and session outcomes; a one-turn retrieval score is not a sufficient statistic.

38.5 Frontend projection

When choices, product cards, citations, or widgets affect task completion, presentation parameters belong in the behavior-complete release. Their interventions require projection and session evidence. The shared optimizer can carry event artifacts without knowing Garden's widget semantics.

39. Trust boundaries and security

39.1 Search is untrusted input

A search policy can propose malformed, unsafe, or misleading candidates. It may understate effect grade, exploit cache keys, or choose a suite that hides regressions. Every proposal is validated by typed adapters and the reducer. Search code runs with no direct ability to activate a release.

39.2 Artifact and evaluator trust

Content identity proves that bytes did not change after hashing. It does not prove the artifact was built from authorized inputs or by the claimed code. Build and evaluation attestations should bind code identity, input manifests, environment, and actor. Remote providers require response and disclosure traces rather than assumed determinism.

39.3 Policy changes

Security policy is versioned input. A previously passing candidate can become inadmissible when threat model or provider policy changes. Evidence certificates therefore carry policy identity and optional expiry. Promotion checks current policy before activation.

39.4 Least authority

Workers receive capability-limited access to the exact candidate, inputs, and output location they require. Evaluators do not activate releases. Search controllers do not write final decisions. Report renderers do not mutate evidence. This operational decomposition mirrors the categorical separation of interfaces.

40. Correctness argument for the architecture

The architecture does not promise absolute correctness of arbitrary RAG behavior. It provides conditional guarantees.

> **Theorem schema 40.1 — Compositional impact soundness.** Assume every atomic intervention has a sound changed-support declaration, every dependency graph edge relation is conservative, and closure is computed correctly. Then the closure of a composite intervention contains every artifact, runtime stage, and evaluator whose denotation may change.
>
> **Proof sketch.** Atomic soundness gives changed roots. Conservative reachability includes every transitive semantic dependency. Closure preserves union, so sequential or parallel composition cannot remove an affected node. ▫
>
> **Theorem schema 40.2 — Evidence monotonicity.** Assume effect grading is conservative and `Req` is monotone. Then composing interventions cannot reduce the minimum evidence required for promotion.
>
> **Proof.** Composition joins or raises grades; monotonicity maps the larger grade to a superset of obligations. ▫
>
> **Theorem schema 40.3 — Promotion authorization.** Assume all referenced evidence certificates are valid for the candidate and current policy, the decision kernel is correctly implemented, and promotion is reachable only through the reducer. Then a promoted candidate has passed every encoded hard constraint and protected comparison required by its declared effect grade.
>
> **Limit.** The theorem does not establish that suites are representative, judges are correct, grades are complete, or external providers obey undeclared semantics.

These conditional schemas identify where formal proof, property tests, code review, empirical evaluation, and human governance must meet.

Part VI. Executable reference sandbox

41. Scope and design of the sandbox

41.1 Purpose

The sandbox exists to answer a concrete question: can the proposed mathematical decomposition be implemented as ordinary, inspectable Go packages and used to optimize both indexing and querying in one campaign? It is not a production benchmark and does not call an LLM. It deliberately avoids third-party dependencies so that every semantic mechanism is visible in the source bundle.

The code base contains 42 Go files, approximately 3,743 nonblank lines, and 29 test functions across fifteen package directories. It compiles with Go 1.23 and uses only the standard library. Reproduction requires no network, database, model provider, or hidden service.

41.2 Package map

The implementation follows the dependency direction developed in the thesis.

- `internal/canon` implements canonical JSON and domain-separated SHA-256 identities.
- `pkg/algebra` implements generic monoids and finite law checks.
- `pkg/kernel` implements finite probability distributions and Markov kernels.
- `pkg/optic` implements cartesian lenses and their laws.
- `pkg/change` implements actions, derivatives, finite sets, DAG closure, and reuse.
- `pkg/field` implements effect grades, evidence obligations, interventions, spaces, and reindexing.
- `pkg/experiment` implements fidelities, exact pairing, and finite Blackwell witnesses.
- `pkg/metric` implements mergeable statistics, intervals, and Pareto order.
- `pkg/decision` implements ordered, interval-aware promotion.
- `pkg/campaign` implements events, reduction, invariants, logs, and state-space exploration.
- `pkg/ragtoy` implements a complete hybrid retrieval system and optimizer.
- `pkg/lawcheck` assembles executable witnesses.

Commands expose the campaign, law report, and model checker. A companion TLA+ module expresses the campaign transition system and core invariants.

41.3 Trust boundary in code

The campaign search loop can compute arbitrary allocation scores, but it cannot construct a promoted state directly. `campaign.Reduce` is the only state transition function. `decision.Evaluate` is the only source of passing decisions in the demo. `field.Required` is evaluated before hard gates. This organization makes the “no scalar bypass” proposition a code property rather than a documentation convention.

42. Executable algebraic kernels

42.1 Finite Markov kernels

`pkg/kernel` represents a distribution as a finite map from values to probability mass. It supplies normalization, Kleisli bind, deterministic embeddings, composition, tensor, copy, discard, total variation, and approximate equality. Generic comparable Go types represent finite objects.

The implementation is small enough to inspect. Its role is not numerical performance; it demonstrates that paired experimental diagrams and Blackwell factorization can be expressed in the same compositional base used by the thesis.

42.2 Lenses

`pkg/optic.Lens[S,A]` has `Get` and `Put`. `Compose` focuses a subpart inside a part. `CheckLaws` evaluates get–put, put–get, and put–put over finite samples. `Commute` checks finite evidence that two updates commute.

The field package builds atomic replacements from a lens, a target node, an effect grade, a hypothesis, and a value. Sequential composition joins targets and grades and concatenates the atomic path.

42.3 Change actions and graphs

`pkg/change.Action` supplies zero, combination, and action. `Differential` supplies a forward map and derivative. `CheckDerivative` verifies the incrementalization square over finite samples.

The DAG implementation rejects cycles, returns deterministic topological order, computes downstream closure, and renders DOT. The reuse predicate is exactly support disjointness. External identity checks remain the responsibility of artifact or experiment code, matching Proposition 12.1.

42.4 Effects and obligations

Effects and obligations are bitset representations of finite join-semilattices. Bitwise OR is join. The `Required` function is monotone by construction; a test exhaustively checks the finite basis. Human-readable ordered names preserve stable reports.

42.5 Typed spaces and reindexing

A finite `Space[A]` supports enumeration, mapping, product, dependent bind, and filtering. The toy candidate space is generated from typed finite values and validated, rather than decoded from arbitrary maps. `Reindex[A,B]` maps both specifications and support sets; composition and identity are tested.

43. The hybrid RAG system

43.1 Corpus and suite

The sandbox contains twelve small documents in three domains: transit operations, gardening, and RAG systems. The twelve evaluation cases include exact lexical questions, semantic paraphrases, and a security-sensitive authorization question. Each case contains relevant document identities and answer-support terms.

The corpus is intentionally transparent. It is large enough to exercise chunk boundaries, synonym-like semantic normalization, lexical/vector disagreement, approximate vector search, fusion, reranking, and context truncation. It is not intended to estimate real product quality.

43.2 Behavior-complete specification

The toy release specification is:

```
type Spec struct {
    ChunkWords     int
    ChunkOverlap   int
    EmbeddingDim   int
    VectorMode     VectorMode // exact or fast
    CandidateDepth int
    FusionAlpha    float64
    Rerank         bool
    ContextK       int
}
```

`Spec.ID` identifies full behavior. `BuildKey` hashes only chunk words, overlap, and embedding dimension. This allows the campaign to demonstrate safe build reuse for query-only candidates.

The baseline uses eighteen-word non-overlapping chunks, sixteen-dimensional embeddings, approximate vector search, depth four, fusion weight 0.55, no reranking, and context count two.

43.3 Build interpreter

The build stage tokenizes documents, creates fixed-size chunks with overlap, computes lexical term counts and document frequencies, and creates deterministic hash embeddings over a small semantic-normalization vocabulary. It records documents, chunks, tokens, embedding dimension, modeled build units, and storage units.

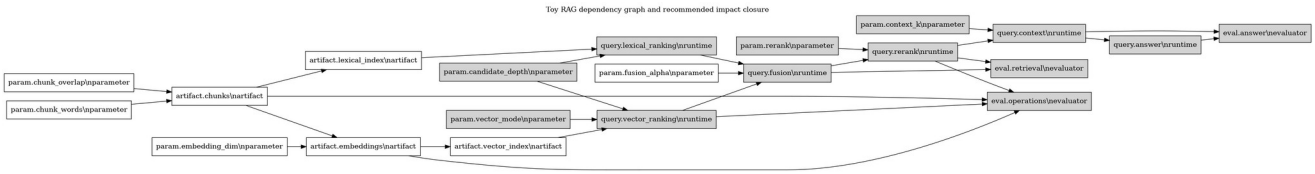
Chunk and embedding parameters determine artifact bytes. Query parameters do not. The build cache is keyed by the build projection and verified by tests.

43.4 Query interpreter

A query produces lexical and semantic tokens. Lexical candidates are ranked with a BM25-style score. Vector candidates are ranked by cosine similarity. Exact mode scans all chunks. Fast mode deterministically subsamples chunks from the paired seed and adds bounded seeded score noise. Both channels retain a configured depth.

Weighted reciprocal-rank fusion combines the channels. Optional reranking uses semantic overlap. The top `ContextK` chunks form context. The trace records channel counts, scanned vectors, fused ranking, context, and a deterministic modeled latency.

The answer evaluator does not generate prose. It measures whether expected support terms are present in context. This keeps the experiment focused on retrieval and context semantics while still exposing a downstream answer-support coordinate.



The toy graph includes build artifacts, runtime ranking stages, and evaluator supports. Highlighting identifies the recommended candidate's impact closure.

43.5 Metrics

Every paired cell produces:

- `recall_at_context`;
- mean reciprocal rank;
- fraction of expected answer terms supported by context;
- modeled latency;
- build and storage units;
- agreement with exact-vector oracle;
- a security-compliance sentinel.

The security sentinel is always one because the sandbox has no private data or remote provider. It demonstrates gate shape only and must not be interpreted as a security test.

44. The candidate field and impact plans

44.1 Candidate enumeration

The finite search space varies:

- chunk words in $\{8, 12, 18\}$;
- overlap in $\{0, 2\}$;
- embedding dimension in $\{16, 32\}$;
- vector mode in $\{exact, fast\}$;
- depth in $\{4, 6\}$;
- fusion alpha in $\{0.35, 0.55, 0.75\}$;
- reranking on or off;
- context count in $\{2, 3\}$.

After validation this yields 576 specifications, including the baseline, so 575 candidates are evaluated.

44.2 Intervention extraction

`ragtoy.Diff` compares baseline and candidate. Each changed field maps to a typed dependency node and minimum effect:

- chunk size, overlap, and embedding dimension: knowledge;
- vector mode: approximation;
- depth, fusion, and reranking: relevance;
- context count: relevance plus interaction.

The compound grade is the join. The atomic path is retained as stable textual declarations for the demonstrator. A production version would use typed atomic candidate references.

44.3 Closure and build requirement

The graph contains eight parameter nodes, four artifact nodes, six query stages, and three evaluator nodes. Closure computes the affected set. A candidate requires build when its impact intersects the artifact-node set. Evaluators whose support is disjoint are reported as reusable.

The recommended candidate changes vector mode, depth, reranking, and context count. Its closure reaches vector ranking and downstream stages but not chunk, lexical, embedding, or vector-index artifact bytes. It therefore reuses the baseline build lawfully.

45. Campaign design and results

45.1 Fidelity chain

The campaign has three fidelities.

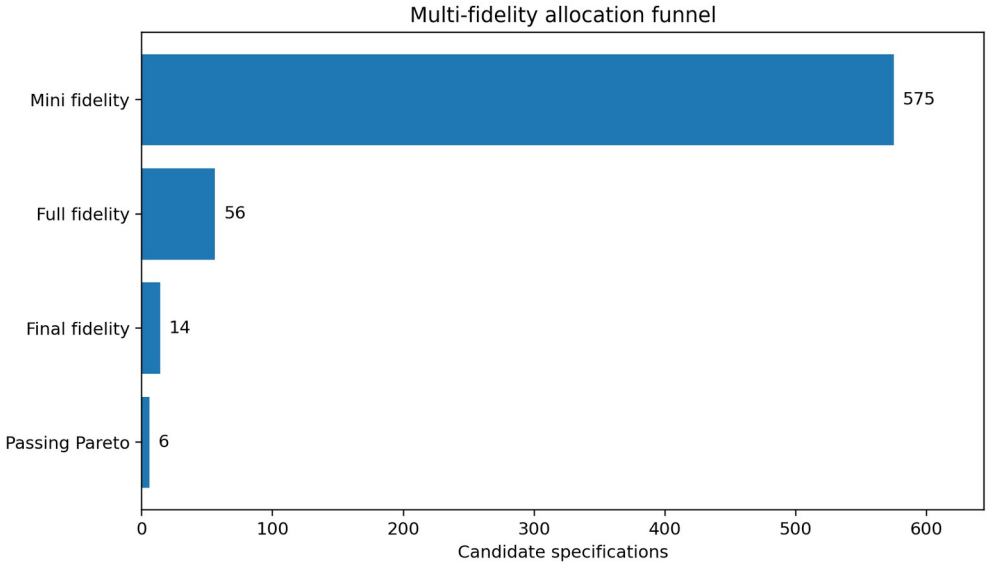
Fidelity	Cases	Repeats	Cost weight	Purpose
mini	6	1	1	broad allocation
full	12	3	3	stable ranking of survivors
final	12	16	12	interval-aware promotion

The chain is ordered by rank and validated. It is not claimed that the mini suite is a formal Blackwell garbling of final evaluation; it is an allocation heuristic. The implementation separately demonstrates an exact finite Blackwell witness on a constructed experiment.

45.2 Allocation policy

All 575 candidates receive mini evaluation. A scalar combines MRR, context recall, answer support, oracle agreement, latency, and build units to rank allocation. The highest 56 receive full evaluation. The highest 14 receive final evaluation.

The scalar is intentionally visible in source. It is not used by `decision.Evaluate`. This demonstrates the architectural separation between exploratory preference and promotion authority.



The campaign aggressively narrows the field while preserving final promotion gates.

45.3 Promotion policy

The final policy requires:

- complete pairing;
- all obligations derived from the candidate grade;
- answer support lower bound of 0.68;

- latency upper bound of 12.5 modeled milliseconds;
- exact-oracle agreement lower bound of 0.70;
- security sentinel equal to one;
- recall non-inferiority margin of 0.04;
- answer-support non-inferiority margin of 0.03;
- MRR improvement lower bound of 0.005.

Absolute constraints use the conservative interval endpoint. Protected and target metrics use paired-delta intervals.

45.4 Results

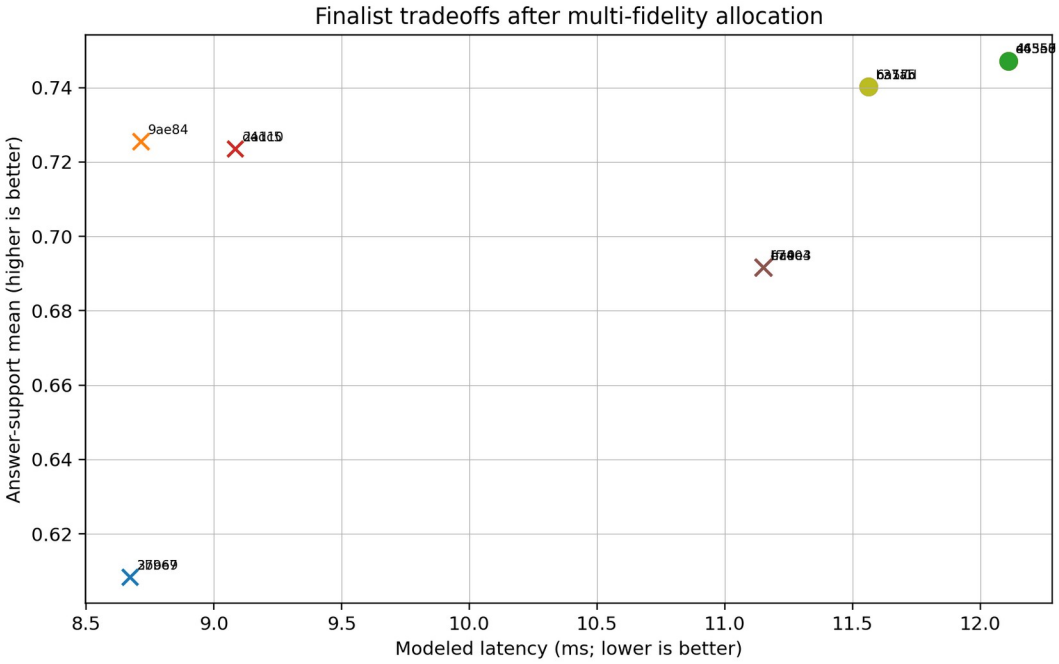
Six finalists pass and are non-dominated under the selected point-estimate objectives. The recommended release is:

```
{
  "chunk_words": 18,
  "chunk_overlap": 0,
  "embedding_dim": 16,
  "vector_mode": "exact",
  "candidate_depth": 6,
  "fusion_alpha": 0.55,
  "rerank": true,
  "context_k": 3
}
```

It has effect grade `approximation + relevance + interaction` and does not require a build. Relative to the paired baseline at final fidelity, the reported means are:

Metric	Baseline	Candidate	Paired delta	95% interval for delta
MRR	0.9036	1.0000	+0.0964	[0.0630, 0.1297]
context recall	0.9427	1.0000	+0.0573	[0.0243, 0.0903]
answer support	0.4378	0.7472	+0.3095	[0.2558, 0.3631]
exact-oracle agreement	0.8737	1.0000	+0.1263	[0.0973, 0.1553]
modeled latency	5.8365	12.1103	+6.2739	[6.2262, 6.3215]
build units	788.6	788.6	0	[0, 0]

The latency increase is substantial but remains within the hard budget. This is precisely the kind of tradeoff a scalar-only optimizer can conceal. The Pareto report exposes it.



Finalists occupy different quality, latency, and build-cost tradeoffs; circles passed the final policy and crosses did not.

45.5 Cache and campaign evidence

The run reports a 99.9 percent build-cache hit rate because many specifications share three build projections. The query-cache hit rate is 54.0 percent because baseline cells and repeated candidate identities are reused under exact keys. These percentages describe this finite campaign, not a performance claim for production.

The event log contains proposed, built when necessary, started/completed at each fidelity, decided, promoted, and rejected records for every finalist. The promoted candidate is active only after a passing decision event.

46. Law checks and model exploration

46.1 Executable law report

`cmd/lawcheck` emits a JSON report covering:

1. identity and associativity of finite Markov-kernel composition;
2. lens laws;
3. the change-action derivative law;
4. monotonicity of effect-to-obligation mapping;
5. monotonicity of dependency closure;
6. paired-seed identity;
7. a finite Blackwell garbling witness;
8. campaign transition invariants.

All checks pass in the included report.

46.2 State-space exploration

`campaign.ModelCheck(9)` begins from empty state, attempts every candidate event for two representative candidates, ignores rejected illegal transitions, and checks invariants in every accepted successor. It explores 477 unique states and 1,496 legal transitions.

The checker is bounded and the state key includes sequence. It is an implementation test, not a complete proof for unbounded candidates or concurrency. The TLA+ model provides a path toward a more abstract exhaustive analysis with fairness and multiple workers.

46.3 Property and integration tests

The Go test suite includes:

- monoid associativity and identity;
- kernel category and tensor laws;
- lens composition and commutativity evidence;
- graph cycle rejection and closure laws;
- intervention category identity/associativity;
- dependent-space cardinality;
- exact pairing and fidelity validation;
- metric merge and Pareto behavior;
- decision pass/fail/need-more cases;
- campaign transition rejection;
- build-identity separation;
- deterministic query traces under equal seeds;
- impact-driven build reuse;
- full campaign promotion.

A production program should add fuzzing, mutation testing, backend differential tests, load tests, and fault injection.

47. Reproduction and artifact layout

The complete sandbox can be reproduced with:

```
cd optfield-sandbox
./scripts/reproduce.sh
```

The script runs all Go tests, law checks, the bounded model checker, the full optimization campaign, Graphviz rendering when available, and checksums. It writes:

```
demo-output/
  campaign.json
  campaign.jsonl
  candidates.csv
  finalists.csv
  pareto.csv
  summary.md
  law-report.json
  modelcheck.json
  dependencies.dot
  dependencies.svg
  dependencies.png
  checksums.sha256
```

`campaign.json` is intentionally large because it preserves the complete paired evidence for all candidates. The CSV files are derived projections, not authority artifacts.

48. Limits of the executable model

The sandbox has deliberate limitations.

First, finite Markov kernels do not model general measurable spaces or continuous provider outputs. Second, semantic token normalization is a hand-authored toy rather than a learned embedding model. Third, latency is modeled, not measured under load. Fourth, answer support is a context-term proxy rather than an LLM outcome. Fifth, the security coordinate is a sentinel. Sixth, normal intervals are used for simplicity. Seventh, effect grades are generated from known fields and do not analyze arbitrary code. Eighth, the TLA+ model is included but TLC was not available in the build environment; only the Go state-space checker was executed.

These limitations do not invalidate the architectural demonstration. They define the boundary of its evidence. The sandbox shows that the abstractions compose, that joint indexing/query search can be executed, that closure drives reuse, and that a search scalar can be prevented from bypassing promotion gates. Production adoption still requires domain-specific interpreters and certificates.

Part VII. Evaluation, research agenda, and conclusion

49. Evaluation against the design criteria

49.1 Compositionality

Atomic interventions compose sequentially. Their paths, target sets, impact closures, effect grades, and evidence obligations compose by concatenation or join. Independent interventions have a certified parallel combinator. Parameterized system composition accumulates local parameter objects rather than flattening them.

49.2 Semantic specificity

The architecture distinguishes operational, approximation, relevance, knowledge, policy/security, interaction, and presentation effects. Observation-indexed equivalence prevents an answer-text comparison from standing in for disclosure or runtime preservation.

49.3 Runtime completeness

Campaign behavior is not left as orchestration. It has events, a partial reducer, safety invariants, replay denotation, and model-check targets. Builds and evaluations are runtime transitions with immutable evidence.

49.4 Statistical integrity

The design makes pairing an identity relation, retains missing cells, distinguishes allocation fidelity from promotion fidelity, and uses interval-valued gates. Blackwell order supplies a precise standard for claims that one fidelity subsumes another.

49.5 Package boundary

RAG meaning remains in `ragkit`; generic optimization custody remains in `ragopt`; the adapter carries typed RAG intervention semantics. Applications retain native objectives and traces. This directly addresses the overlap and abstraction concerns that motivated the study.

50. Open mathematical questions

50.1 A full category of RAG optics

The thesis uses lenses for local configuration and explicit channels for feedback. A fuller theory could model retrieval, evaluation, and controller feedback as parameterized optics in a Markov or enriched setting. Questions include lawfulness under stochastic residuals, state, and partial failure.

50.2 Enrichment by cost and error

Impact and promotion currently use separate structures for dependency, cost, and approximation. An enriched category could attach latency, monetary cost, resource demand, and divergence bounds to morphisms with compositional accumulation laws. The challenge is avoiding false precision when costs are workload-dependent.

50.3 Change actions for probabilistic programs

A semantic derivative for stochastic transducers could produce changes in output distributions and traces, not only support closure. This may connect incremental probabilistic programming, sensitivity analysis, and RAG maintenance. Approximate indexes and provider nondeterminism make the correct equality notion nontrivial.

50.4 Fibred experiment doctrines

Experiments, observations, and obligations vary by architecture. A formal fibration of statistical experiments over system architectures could characterize when evidence transports, when a suite is sufficient after reindexing, and how online experiment fibers relate to offline ones.

50.5 Compositional causal claims

Candidate paths retain causal hypotheses, but the current theory does not prove causal identification. A richer model could connect intervention categories to structural causal models, factorial designs, and interference. The distinction between a configuration intervention and an observational policy change is especially important online.

50.6 Proof assistants

The finite laws could be formalized in Lean, Coq, or Agda. Promising targets include closure algebra, grade monotonicity, reducer safety, replay adequacy, path validation, and support-disjoint reuse. Provider and statistical assumptions would remain axioms or external certificates.

51. Engineering research agenda

A practical research program can proceed in six stages.

1. Add typed intervention descriptors and effect grades to existing atomic `ragopt` candidates.
2. Build a machine-readable RAG dependency graph from `ragkit` specifications and verify closure against known rebuild behavior.
3. Introduce path candidates and support-aware cache admission while retaining current snapshots and runstore.
4. Add fidelity identity, randomization design, interval statistics, and evidence certificates to evaluation artifacts.
5. Put campaign authority behind the event reducer and connect it to the production job system.
6. Run differential campaigns in GEC, RAG-TTC, and Garden, refining the shared field only where at least two applications exhibit the same semantics.

The highest-value early experiment is likely not a sophisticated search algorithm. It is a closure-and-reuse campaign that proves fusion and reranking candidates can share upstream rankings while chunking candidates cannot, with exact artifact identities and paired downstream evaluation.

52. Conclusion

RAG optimization is not the problem of maximizing a score over a configuration dictionary. It is the controlled evolution of an open stochastic service whose knowledge, approximations, policies, runtime behavior, and user interaction can all change. The correct abstraction must retain locality, causality, state, uncertainty, and authority.

The **optimization field** supplies that abstraction. Over each architecture lies a category of behavior-complete specifications and typed interventions. Parameterized stochastic transducers give denotational meaning. Optics focus local changes. Change actions and dependency derivatives explain incremental rebuild and reuse. Effect grading derives monotone evidence obligations. Statistical experiments give finite observations and fidelity relations. Constraint-first partial orders govern promotion. A cybernetic controller proposes work, while a small operational reducer preserves authority and replay.

The mathematical structures are valuable because they identify implementable laws. Identity and composition become code. Closure becomes a scheduler input and a reuse theorem. Pairing becomes an equality of experimental cells. “Strictest concern wins” becomes a semilattice homomorphism. “Search cannot promote” becomes a transition invariant. The accompanying Go sandbox demonstrates these claims in a complete joint indexing/query campaign.

The proposed architecture does not eliminate product judgment or uncertainty. It makes them explicit and locates them at stable interfaces. That is the appropriate foundation for evolving `ragkit`, `ragopt`, and the applied RAG systems into one composable optimization discipline without collapsing their meanings into a second monolith.

Appendices

Appendix A. Formal notation and core definitions

This appendix collects the mathematical objects used throughout the thesis in one place. It is intentionally more explicit than the main text. The purpose is not to force the implementation to expose category-theory terminology. The purpose is to make the interface laws, variance, and proof obligations precise enough that independent implementations can be compared.

A.1 Categories, monoidal structure, and enrichment

A **category** C consists of objects, hom-sets $C(X,Y)$, identity arrows 1_X, and associative composition. The convention in this volume is that $g \circ f$ performs f first. An ordinary deterministic program $f : X \to Y$ is treated as an arrow in a category of value types and total functions. Partiality, failure, state, and probability require richer categories or explicit result objects.

A **symmetric monoidal category** $(C, \otimes, I)$ permits parallel composition. The tensor $X \otimes Y$ represents independent interfaces placed side by side; it does not mean that values may always be duplicated. Associators, unitors, and braidings are omitted from formulas when coherence makes their placement unambiguous.

Several structures in the thesis can be understood as enrichment or decoration of ordinary arrows:

- a cost-enriched arrow carries an upper bound, estimate, or distribution of resource consumption;
- a metric-enriched arrow carries an approximation distance;
- a graded arrow carries a semantic effect;
- a labeled arrow carries a causal hypothesis and audit identity;
- an indexed arrow lives in the fiber belonging to one architecture.

The implementation keeps these structures separate because no single numeric enrichment faithfully represents all of them. For example, semantic security effect has no lawful conversion into milliseconds, and a confidence interval is not a monoidal cost.

A.2 Finite Markov kernels

Let $FinStoch$ denote the finite stochastic category. Its objects are finite sets. An arrow $k : X \rightsquigarrow Y$ is a row-stochastic matrix, equivalently a function

$$k : X \to D(Y),$$

where $D(Y)$ is the set of finite probability distributions over Y. The identity is $\eta_X(x) = \delta_x$. Composition is

$$(\ell \circ k)(x)(z) = \sum_{y \in Y} k(x)(y)\,\ell(y)(z).$$

The tensor is the independent product:

$$(k \otimes \ell)(x,u)(y,v) = k(x)(y)\,\ell(u)(v).$$

Deterministic functions embed by Dirac distributions. Every deterministic data object has copy and discard maps

$$\Delta_X : x \mapsto (x,x), \quad !_X : x \mapsto *,$$

satisfying the commutative comonoid laws. A general stochastic arrow does not preserve copying. This distinction is central to paired evaluation. The case identifier, repeat, and seed design may be copied

before the two candidate arms execute; one realized random output must not be copied and represented as two independent samples.

A production semantics may replace $FinStoch$ with a suitable Markov category over measurable spaces. The finite implementation is a witness that the equations can be executed, not a restriction on the abstract architecture.

A.3 Stateful stochastic transducers

For runtime state S, input X, output Y, and trace Tr, a one-step system is

$$t : S \otimes X \rightsquigarrow S \otimes Y \otimes Tr .$$

Sequential execution threads the state component. Given

$$t : S \otimes X \rightsquigarrow S \otimes Y \otimes Tr_t , u : S \otimes Y \rightsquigarrow S \otimes Z \otimes Tr_u ,$$

their stateful composite first samples t, then feeds its new state and output to u, and combines traces with an associative trace operation. A trace may be a free monoid of events, a structured event tree, or an append-only artifact reference. The relevant law is associativity of trace combination and compatibility with identity.

A system is **open** when its interfaces remain visible under composition. An index builder, retriever, reranker, answer generator, and projector are not forced into one opaque arrow. Wiring data says which ports are connected. This permits architecture-indexed optimization: an intervention can target a port, component parameter, or wiring decision without pretending that all systems share one flat record.

A.4 The parameterized category

Given a symmetric monoidal category C, the parameterized category $Para(C)$ has the same interface objects as C. A morphism $X \to Y$ is represented by a parameter object P and an arrow

$$f : P \otimes X \to Y .$$

Two presentations are identified up to the selected equivalence on parameter objects. Composition of $(P, f) : X \to Y$ and $(Q, g) : Y \to Z$ has parameter object $Q \otimes P$ and behavior

$$Q \otimes P \otimes X \xrightarrow{1_Q \otimes f} Q \otimes Y \xrightarrow{g} Z .$$

For RAG, P includes far more than learned weights. It can contain a corpus barrier, source policy, chunker identity, embedding model, index algorithm, query policy, prompt, provider configuration, timeout, validator, and projection contract. A **behavior-complete** parameter object contains every input whose change can alter the selected observation family.

A configuration field that merely changes logging verbosity may be omitted from an answer-only behavior identity but cannot be omitted from an operations-trace identity. Behavior completeness is therefore indexed by observation scope.

A.5 Lenses and optics

A cartesian lens from a whole S to a focus A is a pair

$$get : S \to A , put : S \times A \to S ,$$

with the laws

$$\begin{aligned} put(s, get(s)) &\quad s, \\ get(put(s, a)) &\quad a, \\ put(put(s, a_1), a_2) &\quad put(s, a_2). \end{aligned}$$

The laws guarantee that focusing and rebuilding a release does not introduce unrelated drift. A focused update $u : A \to A$ lifts to

$$\hat{u}(s) = put(s, u(get(s))).$$

Lenses compose, so a package can expose a focus on `Query.Fusion.Alpha` without exposing the entire release representation. More general optics permit residual context, non-cartesian tensors, and bidirectional feedback. The executable kernel uses lenses because their laws are transparent and sufficient for immutable configuration updates. The abstract architecture leaves room for parameterized optics when evaluator feedback and controller updates are modeled in one compositional object.

A.6 Change actions and derivatives

A **change action** on A consists of a monoid $(\Delta A, +, 0)$ and an action

$$\oplus : A \times \Delta A \to A$$

such that $a \oplus 0 = a$ and $(a \oplus \delta_1) \oplus \delta_2 = a \oplus (\delta_1 + \delta_2)$. A derivative of $f : A \to B$ is a map

$$Df : A \times \Delta A \to \Delta B$$

satisfying

$$f(a \oplus \delta) = f(a) \oplus Df(a, \delta).$$

The derivative need not be linear or real-valued. For a build program, Df may be an artifact delta, a work plan, or a conservative set of invalidated nodes. The thesis uses three levels:

1. **semantic delta**, which exactly reconstructs the new output;
2. **artifact delta**, which identifies bytes or rows that must change;
3. **support derivative**, which returns a conservative downstream node set.

The dependency DAG implements the third. It is sound when every output node that can change after an input change appears in the returned closure. It may over-approximate, causing unnecessary work but not stale reuse.

A.7 Dependency doctrine

For architecture A, let N_A be a finite set of semantic nodes and $\rightsquigarrow_A$ a directed acyclic dependency relation. For $U \subseteq N_A$, define the downstream closure as the least set satisfying

$$U \subseteq cl_A(U), x \in cl_A(U) \wedge x \rightsquigarrow_A y \Rightarrow y \in cl_A(U).$$

The closure operator is extensive, monotone, and idempotent:

$$U \subseteq cl(U), U \subseteq V \Rightarrow cl(U) \subseteq cl(V), cl(cl(U)) = cl(U).$$

For finite DAGs, it also preserves union:

$$cl(U \cup V) = cl(U) \cup cl(V).$$

An artifact or evaluator has a declared support $supp(a) \subseteq N_A$. Support is intensional: it names semantic dependencies, not merely files read during execution. A retriever metric may depend on query

rewriting, ranking, and relevance labels but not on the answer generator. An answer-support metric depends on the selected context and therefore on every upstream retrieval stage.

A.8 The intervention category

Fix architecture A. The objects of $Opt(A)$ are valid behavior-complete specifications $\theta \in \Theta_A$. A morphism

$$i : \theta \to \theta'$$

contains:

- a total, validated update from θ to θ';
- an ordered atomic path identity;
- a primitive target support $supp(i)$;
- a semantic effect grade $g(i)$;
- a causal hypothesis;
- optional preservation claims;
- a schema and interpreter version.

Identity performs no update, has empty support, bottom grade, and empty path. Sequential composition concatenates paths, composes updates, unions target support, joins grades, and intersects preservation claims. The endpoint is validated after composition.

For a fixed baseline θ_0, candidates form the coslice category

$$(\theta_0 \downarrow Opt(A)).$$

An object is an arrow $i : \theta_0 \to \theta$. A morphism from i to j is an extension k with $j = k \circ i$. This captures progressive campaigns: a candidate is not merely a configuration but a located intervention history from the baseline.

A.9 Effect grading

Let G be a finite join-semilattice generated by semantic concerns:

$$\{op, approx, rel, know, policy, interact, present\}.$$

The implementation represents a grade as a set of generators and join as union. The grading map

$$g : Mor(Opt(A)) \to G$$

satisfies

$$g(1_\theta) = \bot, \; g(j \circ i) = g(i) \vee g(j).$$

This is a conservative grade. A compound path never becomes less consequential than either component. Product-specific grades may refine the shared generators, provided there is a monotone forgetful map into the common lattice.

A.10 Evidence doctrine

Let E be a join-semilattice of evidence obligations. The map

$$Req : G \to E$$

is monotone and preferably join-preserving:

$$Req(g_1 \vee g_2) = Req(g_1) \vee Req(g_2).$$

An evidence certificate is not just a set bit. It should record the artifact identity, producer, suite, fidelity, source barrier, evaluator version, randomization design, timestamps, and verification outcome. The finite implementation uses a bitset to make the algebra visible; production code must attach native artifacts to each obligation witness.

The doctrine is indexed by architecture and product policy. A common approximation grade may require exact-oracle comparison everywhere, while a medical product may add a domain review and a customer-facing product may add session calibration. Reindexing an intervention must reindex or strengthen its obligations.

A.11 Architecture indexing and the Grothendieck construction

Let A be a category whose objects are RAG architectures and whose arrows are architecture mappings. An arrow $r : B \to A$ can represent restriction to a subsystem, an embedding of a shared component, a migration, or a semantics-preserving translation. The optimization assignment is contravariant:

$$Opt : A^{op} \to Cat.$$

Thus r induces a reindexing functor

$$r^{\square} : Opt(A) \to Opt(B).$$

Reindexing maps specifications, parameter references, supports, effects, and observation claims. It must preserve identities and composition. Strict preservation may be relaxed to coherent isomorphism in a pseudofunctorial implementation.

The Grothendieck construction $\int Opt$ has objects (A, θ) and morphisms combining an architecture map with a vertical intervention in the target fiber. This total category is the mathematical version of a registry containing several RAG products without flattening their parameter spaces. `ragopt` can manage total-category custody while each `ragkit` or application adapter supplies a fiber.

A.12 Observation families and equivalence

An observation family O selects what is semantically visible. It may include final answers, ranked evidence, traces, latency, cost, disclosure, frontend events, or session outcomes. For release θ, let

$$Obs_O(\theta) : C_O \rightsquigarrow Y_O$$

be the induced stochastic channel from controlled context to observations. Exact O-equivalence is equality of channels:

$$\theta \equiv_O \theta' \Leftrightarrow Obs_O(\theta) = Obs_O(\theta').$$

Approximate equivalence relative to a divergence d_O and tolerance ε is

$$\theta \approx_{O,\varepsilon} \theta' \Leftrightarrow^{\square} c \in C_O \, d_O\big(Obs_O(\theta)(c), Obs_O(\theta')(c)\big) \le \varepsilon.$$

Finite evaluation cannot usually establish this universal statement. It supplies evidence about a suite-indexed restriction. Preservation claims must therefore name the observation family, domain, tolerance, and evidence scope.

A.13 Statistical experiments

A statistical experiment is a channel from a latent condition or controlled context to an observation. For release θ, fidelity f, case c, and deterministic randomization context z:

$$E_{\theta,f}(c,z) \in D(O_f).$$

A paired contrast uses the copying map on (c,z):

$$(c,z) \xrightarrow{\Delta} \big((c,z),(c,z)\big) \xrightarrow{E_{\theta_0,f} \otimes E_{\theta_1,f}} (o_0,o_1).$$

The pairing key is an identity relation, not a join heuristic. If either arm is missing, the cell remains incomplete unless the protocol explicitly defines censoring or imputation.

Experiment H is at least as informative as L in the Blackwell order when there exists a channel G with

$$L = G \circ H.$$

The channel G is a garbling witness. Case-count inclusion alone does not establish Blackwell order because different prompts, providers, label policies, or sampling processes can change the experiment in incomparable ways.

A.14 Metrics and decision relations

A mergeable metric summary is a commutative monoid $(M,\oplus,0)$ together with an extractor from observations. Distributed evaluation is lawful when partitioning and merge order do not change the summary. Mean statistics use count, sum, and sum of squares; ranking statistics may use sufficient count vectors or retained per-case observations.

A metric estimate is interval-valued:

$$\hat{m} = (\mu, L, U, n, method).$$

A promotion policy defines a feasible predicate over evidence, absolute constraints, protected non-inferiority relations, and a target improvement. The three-valued decision set is

$$V = \{pass, fail, need\text{-}more\}.$$

The third value is semantically important. An interval crossing a threshold is not a weak pass or weak fail; it is insufficient evidence under that policy.

After hard feasibility, candidates are compared by a product order with explicit metric directions. A candidate is Pareto dominated when another feasible candidate is no worse on every selected coordinate and strictly better on at least one. The frontier is a report, not an automatic release choice.

Appendix B. Laws, propositions, and proof obligations

This appendix states the principal correctness claims in theorem-like form. Most proofs are straightforward once assumptions are explicit. The engineering difficulty lies in ensuring that concrete identities, dependency supports, and interpreters satisfy those assumptions.

B.1 Category laws for finite kernels

Proposition B.1 (finite stochastic category). Normalized finite distributions with Dirac identity and Kleisli composition form a category.

Proof sketch. Left and right identity follow from multiplication by the point mass. Associativity follows by rearranging a finite triple sum:

$$\sum_y k(x)(y)\sum_z \ell(y)(z)m(z)(w)=\sum_z\left(\sum_y k(x)(y)\ell(y)(z)\right)m(z)(w).$$

Nonnegativity is preserved and total mass remains one. The executable law check compares both association orders on finite kernels within a numerical tolerance.

Production obligation. Any optimized distribution representation must preserve normalization, reject NaN and negative masses, and state the tolerance used for approximate equality. Provider APIs that return truncated alternatives are subprobability observations until an explicit “other” outcome or renormalization policy is supplied.

B.2 Tensor and deterministic copying

Proposition B.2 (parallel composition). Independent product of finite kernels is bifunctorial:

$$(k_2\circ k_1)\otimes(\ell_2\circ\ell_1)=(k_2\otimes\ell_2)\circ(k_1\otimes\ell_1).$$

Proof sketch. Both sides expand to the product of two independent finite sums. The result depends on the independence represented by tensor. Shared provider rate limits or correlated random generators belong in explicit shared state, not in a false independent tensor.

Corollary B.2.1 (paired context). Copying deterministic context before two stochastic arms is lawful. Copying one stochastic realization and treating the copies as independent is not.

This corollary is the semantic reason the evaluator copies case/repeat/seed identity and executes each arm separately.

B.3 Lens lifting

Proposition B.3 (focused update identity). If $L:S\leftrightarrow A$ is a lawful lens and $u=1_A$, then the lifted update $\hat{u}$ is 1_S.

Proof. $\hat{u}(s)=put(s,get(s))=s$ by get-put.

Proposition B.4 (focused update composition). For ordinary updates $u,v:A\to A$,

$$\widehat{v\circ u}=\hat{v}\circ\hat{u}.$$

Proof sketch. Expand both sides and use put-get and put-put. This law permits a path of atomic focus updates to be interpreted without serializing and reparsing the whole release at every step.

Production obligation. Generated lenses over versioned schemas must be tested across all representable values or derived from a construction that proves the laws. A lens that normalizes unrelated fields during `Put` is not lawful for causal optimization because it creates hidden targets.

B.4 Intervention category

Proposition B.5 (category of interventions). Valid specifications and validated intervention paths form a category under the composition defined in Appendix A.

Proof sketch. The identity update is neutral under function composition, union with empty support, grade join with bottom, and path concatenation with the empty path. Associativity follows from associativity of function composition, set union, join, and list concatenation. Preservation claims use set intersection, which is associative and has the universal claim set as identity. Validation must be deterministic and depend only on the resulting specification and declared schema versions.

Caveat. If candidate identity is computed only from endpoint bytes, associativity of semantic identity still holds, but causal identity is quotiented away. The thesis intentionally keeps path identity.

B.5 Grade conservation

Proposition B.6 (strictest concern wins). The effect map is a grading functor into the one-object category induced by join semilattice G:

$$g(1) = \bot\,, g(j \circ i) = g(i) \vee g(j).$$

Consequently $g(i) \leq g(j \circ i)$ and $g(j) \leq g(j \circ i)$.

Engineering consequence. A relevance-changing intervention followed by an operational intervention cannot be filed as “operational only.” Any path optimizer that permits grade override after composition violates the algebra.

B.6 Monotone evidence

Proposition B.7 (evidence monotonicity). If $Req : G \to E$ is monotone, then

$$g(i) \leq g(j) \Rightarrow Req(g(i)) \leq Req(g(j)).$$

If it preserves joins, required evidence for a path is exactly the join of obligations for its generators.

Proof. Immediate from monotonicity or the homomorphism law.

Production obligation. The obligation schema and policy version are part of campaign identity. Updating policy after an evaluation may strengthen obligations; old evidence can be transported only if a verifier certifies it under the new schema.

B.7 Closure algebra

Proposition B.8 (least downstream closure). For a finite DAG, reachability from U is the least downstream-closed set containing U.

Proof sketch. Breadth-first or depth-first reachability is downstream closed and contains U. Any downstream-closed superset containing U must contain every node reachable by induction on path length.

Proposition B.9 (union preservation).

$$cl(U \cup V) = cl(U) \cup cl(V).$$

Proof. A node is reachable from $U \cup V$ exactly when it is reachable from either U or V.

Corollary B.9.1. Impact closure of a composed intervention can be computed by joining the closures of its atomic target sets. Incremental computation may cache closure per generator.

Caveat. Dynamic dependencies, data-dependent routing, and provider fallbacks can make a static DAG incomplete. A sound production graph may require conditional edges labeled by guards; conservative union over feasible guards is safe but less precise.

B.8 Build-key projection

Let $B_A \subseteq N_A$ be the support of build bytes. Define a build-key projection $\pi_B : \Theta_A \to K_B$ that includes exactly those parameter identities on which the build interpreter depends.

Proposition B.10 (build reuse by projection). If the build interpreter factors as

$$Build = \overline{Build} \circ \pi_B$$

and $\pi_B(\theta) = \pi_B(\theta')$, then build outputs are equal under the build's declared deterministic equality.

Proof. Substitution through the factorization.

Engineering consequence. Full release identity and build identity must be distinct. Query-only candidates may share build bytes without pretending to be the same release.

B.9 Support-disjoint reuse theorem

Let intervention $i : \theta \to \theta'$, impact $I = cl(supp(i))$, and artifact/evaluator a with support S_a. Let ξ denote external identities not represented in the architecture graph, such as corpus barrier, suite, seed design, provider version, and fidelity.

Theorem B.11 (reuse soundness). Suppose:

1. the support declaration is sound: the denotation of a factors through the projection to S_a and ξ;
2. the dependency closure is sound;
3. $I \cap S_a = \varnothing$;
4. external identities agree, $\xi = \xi'$.

Then the artifact or observation produced by a is unchanged under i and may be reused.

Proof sketch. Sound closure implies every node in S_a is unchanged because none lies in the impact set. Support factorization then gives equal inputs to a. Equal external identities provide the remaining arguments. Determinism gives equal artifacts; for a stochastic evaluator, equal channels give equality in distribution under the same randomization design.

Why content hashing alone is insufficient. A cached file can have the same bytes while being semantically invalid under a changed suite, policy, or provider contract. Conversely, a different release identity may lawfully reuse the same upstream rankings. The theorem requires semantic support and external identity, not only byte equality.

B.10 Independent interventions

Two interventions i and j are **structurally independent** when their primitive targets are disjoint. They are **operationally independent** when their impact closures do not contend for noncommutative resources or require incompatible build transactions. They are **behaviorally commuting** when

$$[\![j \circ i]\!]_O = [\![i \circ j]\!]_O$$

for the declared observation family O.

Proposition B.12 (certified parallel composition). If focused updates commute on specifications, effect and target combination are commutative, and the interpreters are behaviorally commuting on O, then either sequential order denotes the same O-behavior and may be represented by a parallel intervention $i \otimes j$.

Caveat. Disjoint fields do not imply behavioral independence. Candidate depth and reranker threshold may occupy different fields but interact through candidate availability. The sandbox therefore requires an explicit commutativity certificate rather than inferring parallelism from target inequality alone.

B.11 Pairing integrity

Let a paired cell key be

$$p = (suite, case, repeat, seedDesign, fidelity, policy).$$

Proposition B.13 (contrast invariance under execution order). If both arms are evaluated from the same deterministic p, evaluator state is either isolated or explicitly modeled, and the metric contrast is a function of the completed pair, then interleaving or execution order does not change the contrast distribution.

Proof sketch. The two-arm diagram factors through copied deterministic context and tensor product. Scheduling is observationally irrelevant under isolation. If there is shared mutable provider or cache state, that state must be added to the experiment; otherwise the assumption fails.

Production obligation. Randomization may intentionally randomize arm order to control temporal confounding. The selected order is then part of p and the analysis design.

B.12 Blackwell transport

Proposition B.14 (decision transport under garbling). If $L = G \circ H$, every decision rule d_L based on low-fidelity observations has a high-fidelity counterpart $d_H = d_L \circ G$ with the same risk under corresponding states.

This is the operational value of a Blackwell witness: high-fidelity evidence can reproduce every low-fidelity decision. The reverse need not hold. The sandbox checks exact matrix factorization for a finite constructed example.

Non-theorem. A suite with more cases is not automatically more informative. If the large suite changes labels or uses a lower-quality provider, there may be no garbling relation in either direction.

B.13 Monoidal reduction

Proposition B.15 (partition-independent summary). Let $(M, \oplus, 0)$ be a commutative monoid and $e : O \to M$ an extractor. For any partition of observations $D = D_1 \uplus \cdots \uplus D_k$,

$$\oplus_{o \in D} e(o) = \oplus_{r=1}^{k} \left(\oplus_{o \in D_r} e(o) \right).$$

Thus distributed workers may merge summaries without changing the result.

Caveat. Quantiles, bootstrap intervals, and ranking metrics may require richer summaries or retention of cell-level data. Calling a non-associative floating-point reducer "monoidal" without a declared numerical tolerance is an overstatement. The production artifact should record reduction order or use stable algorithms when exact reproducibility matters.

B.14 Ordered gate soundness

Let the gate phases be obligation completeness, pair completeness, absolute safety, protected non-inferiority, and target improvement.

Proposition B.16 (no target override). If the gate reducer stops on `fail` or `need-more` before the target phase, no target improvement can convert an infeasible candidate into `pass`.

Proof. By control-flow construction of the ordered decision algebra.

Proposition B.17 (conservatism under interval widening). For one-sided gates defined by interval containment, widening a confidence interval cannot turn `fail` or `need-more` into `pass` unless the widened interval still lies entirely in the passing region. In the usual widening relation, a previous pass may become need-more, but uncertainty cannot manufacture evidence.

B.15 Pareto invariance

Proposition B.18 (positive monotone reparameterization). Pareto dominance is invariant under coordinatewise strictly monotone transformations respecting each metric direction.

Proof sketch. Strictly monotone maps preserve all pairwise order relations on each coordinate.

This permits reporting latency in milliseconds or seconds without changing the frontier. It does not permit mixing hard constraints into a weighted scalar because scalarization can reverse incomparable tradeoffs.

B.16 Campaign reducer safety

Let $R : S \times E \rightharpoonup S$ be the partial reducer.

Theorem B.19 (inductive safety). If the initial state satisfies invariants I, and every accepted transition from an I-state returns an I-state, then every finite accepted event trace reduces to an I-state.

Proof. Induction on trace length.

The implemented invariants include:

- a promoted candidate has a passing decision;
- a candidate is not both promoted and rejected;
- a build-required candidate is not evaluated before build completion;
- an evaluation is not completed before it starts;
- the active candidate is promoted;
- event sequence numbers are contiguous.

The bounded model checker explores every generated legal and illegal action up to depth nine for a finite candidate model and validates reducer behavior. This is not a proof for unbounded campaigns or all production event schemas.

B.17 Replay adequacy

Let $e_1 \cdots e_n$ be an append-only event trace and

$$fold_R(s_0, e_1 \cdots e_n)$$

its left fold through the partial reducer.

Proposition B.20 (prefix replay). If online command handling persists an accepted event before exposing the resulting state, then replaying the durable prefix after a crash yields the same authoritative state as the online reducer at that prefix.

Proof sketch. Determinism of R and induction over the persisted event sequence. Atomic persistence is an external assumption.

Production obligation. Side effects such as build submission and provider calls require an outbox, inbox, or idempotency protocol. Reducer replay alone does not make external effects exactly once.

B.18 Promotion authorization

Theorem B.21 (promotion kernel). Assume candidate identity, effect grade, required obligations, experiment identity, gate policy, and decision artifact are all verified by the trusted kernel. If the reducer accepts `Promoted(c)`, then candidate c has a passing decision under the recorded policy and cannot already be rejected.

This theorem is intentionally narrow. It does not state that the suite represents future users, that an evaluator is valid, or that the product owner should choose the candidate. It states that the runtime cannot bypass the declared evidence and decision process.

B.19 Adequacy triangle

For an interpreter I, operational transition system $\rightarrow$, and projection π from completed traces to denotational observations, the desired adequacy relation is

$$[\![I(\theta)]\!] = \pi_{\square} Exec(\theta),$$

where $\pi_{\square}$ is distribution pushforward. Evaluation then samples a controlled restriction of this channel. A complete correctness argument has three parts:

1. **operational-denotational adequacy** of the interpreter;
2. **experiment provenance** tying samples to interpreter identity and controlled context;
3. **decision adequacy** tying promotion to certified evidence.

Category theory organizes the first and compositional aspects of the second. It does not erase statistical or systems assumptions.

Appendix C. Operational semantics of an optimization campaign

C.1 Runtime configuration

A campaign configuration is

$$C=(n,\Theta,W,B,X,D,A,L),$$

where:

- n is the next event sequence position;
- Θ maps candidate identity to immutable intervention manifests;
- W is queued and in-flight work;
- B maps candidate/build keys to verified build artifacts;
- X maps experiment cell identities to execution state;
- D maps candidates to decision artifacts;
- A is the currently active release or candidate reference;
- L is the durable append-only log.

The minimal sandbox reducer stores a projection of this configuration. Production state should retain native artifact references outside the reducer state and keep only immutable digests, status, and custody references inside it.

A command is a request such as `Propose`, `RecordBuild`, `StartEvaluation`, `CompleteEvaluation`, `Decide`, `Promote`, or `Reject`. A command handler verifies preconditions and emits one or more events. Only events enter the authoritative reducer.

C.2 Proposal rule

A proposal is accepted only when the manifest is canonical, its baseline exists, its path verifies, its endpoint is valid, and its identity is unused:

$$\frac{verifyPath(i)=\theta\, id(i)\notin dom(\Theta)}{C \xrightarrow{propose(i)} C[i\mapsto proposed]}.$$

The transition records the effect grade, target support, required obligations, build-key projection, and claimed preservation relations. Impact closure is recomputed by the trusted dependency doctrine; a proposer-supplied closure is advisory only.

C.3 Build rules

If $cl(supp(i))$ intersects build support, the candidate requires a build. Scheduling and completion are distinct:

$$\frac{status(i)=proposed \quad requiresBuild(i)}{C \xrightarrow{scheduleBuild(i,k)} C[W\cup\{k\mapsto queued\}]}$$

and

$$\frac{W(k)=running\,verifyBuild(a,k)}{C \xrightarrow{buildComplete(i,a)} C[B(i):=a]}.$$

A cache hit is modeled as a verified completion whose artifact already exists. It is not a skipped transition. The verifier checks build key, corpus barrier, derivation versions, manifest digest, and artifact integrity.

When no build is required, the release interpreter references the baseline build plus the candidate's query-time specification. This distinction is necessary for joint indexing/query optimization.

C.4 Evaluation start

An evaluation may start when the required build is present, the experiment manifest is valid, and no cell with the same identity is already terminal:

$$\frac{ready(i,f,p)\; p \notin terminal(X)}{C \xrightarrow{evalStart(i,f,p)} C[X(p):=running]}.$$

The cell identity p includes candidate, baseline, suite, case, repeat, seed design, fidelity, evaluator, and policy versions. Retried attempts have separate attempt identities beneath the same semantic cell. Exactly one accepted result is selected by a deterministic completion rule; duplicates remain auditable.

C.5 Evaluation completion and partial evidence

Completion records a native observation artifact and a verified summary reference:

$$\frac{X(p)=running\,verifyObs(o,p)}{C \xrightarrow{evalComplete(p,o)} C[X(p):=complete(o)]}.$$

Failure and cancellation are explicit terminal attempt outcomes. They do not silently remove a pairing cell. A protocol may reschedule a new attempt, mark the semantic cell censored with a reason, or leave it missing. The decision kernel sees the resulting completeness relation.

C.6 Evidence certification

An obligation witness is certified when its verifier accepts an artifact under the candidate and policy identities:

$$\frac{q \in Req(g(i))\; verify_q(a,i,p)=ok}{C \xrightarrow{certify(i,q,a)} C[evidence(i,q):=a]}.$$

A witness may discharge several obligations only when its verifier explicitly says so. For example, an exact-retrieval comparison may provide both an approximation oracle and retrieval evaluation evidence, but it does not constitute a security review.

C.7 Decision rule

The decision command constructs a closed evidence bundle:

$$E_i=(i,Req(g(i)),certs(i),pairs(i),estimates(i),policy).$$

The gate is a deterministic function

$$Gate : E_i \to V \times Checks.$$

A decision event includes the evidence-bundle digest, policy digest, verdict, and ordered checks. Recomputing the gate from the same bundle must produce the same artifact.

C.8 Promotion and rejection

Promotion is permitted only after `pass`:

$$\frac{D(i) = pass \quad \neg rejected(i)}{C \xrightarrow{promote(i)} C[A := i, promoted(i) := true]}.$$

Production activation may be a second state machine with staged, canary, active, draining, rolled-back, and retired states. The campaign's `promote` event authorizes entry into that activation protocol; it need not perform the deployment itself. The activation result is fed back as a separate observation.

Rejection is terminal for the candidate path under the current campaign identity. A modified path or new evidence policy creates a new candidate or campaign rather than mutating history.

C.9 Illegal transitions

The reducer rejects at least the following:

- duplicate proposal identity;
- build completion for an unknown or build-free candidate;
- evaluation before a required build;
- empty or inconsistent fidelity identity;
- duplicate start or duplicate semantic completion;
- completion before start;
- decision before any completed evaluation;
- invalid verdict value;
- promotion without pass;
- promotion after rejection;
- rejection after promotion;
- any event with a noncontiguous sequence number.

Rejection of an event is not the same as campaign rejection of a candidate. Invalid runtime input is logged as a command failure outside the authoritative event stream or as a distinct administrative event schema.

C.10 Crash semantics

The durable log is authoritative. A safe command path is:

1. read current state and expected sequence;
2. validate the command against that state;
3. append the event atomically with compare-and-swap on sequence;
4. reduce or materialize the new state;
5. dispatch external work through an outbox tied to the event.

After a crash, the service reconstructs state by replaying the log and resumes undispatched outbox items. Workers use idempotency keys based on work identity and attempt identity. Completion is accepted only once per semantic cell, but all attempts remain in native logs.

C.11 Concurrency semantics

Concurrent commands are serialized by sequence compare-and-swap or a transactional stream. This gives a total order for authority events without requiring all work to execute serially. Builds and evaluations may run concurrently when their resource and dependency plans permit.

Parallel candidate evaluation does not imply parallel intervention composition. The former is scheduler concurrency over independent immutable releases. The latter is a semantic claim that two updates commute. These must remain separate types.

C.12 Fairness and liveness

Safety is reducer-local; liveness depends on the scheduler and environment. Useful fairness assumptions include:

- every queued, feasible work item is eventually attempted;
- retryable failures are retried within policy bounds;
- terminal worker results are eventually delivered;
- a candidate with complete required evidence is eventually decided;
- a passing promotion proposal is eventually either activated or explicitly failed.

No system can guarantee these under permanent provider failure or exhausted capacity. The campaign should therefore expose stalled states and deadlines as observations rather than claiming unconditional liveness.

C.13 Online experiments

Online canaries live in a different experiment fiber from offline evaluation. Their context includes traffic allocation, user population, release epoch, interference, temporal window, and privacy policy. A promotion from offline to canary is an architecture or stage mapping with new evidence obligations, not merely a higher numeric fidelity.

The online controller must not mutate the active release in place. It proposes a new immutable release or routing policy, obtains authorization, and changes traffic through the activation protocol. This prevents feedback observations from losing the identity of the behavior that generated them.

Appendix D. Reference API blueprint

The following API is a design blueprint rather than a required public surface. It separates mathematical roles so implementations can evolve independently. The executable sandbox contains a smaller finite version.

D.1 Stable identities

```
type Digest string

type ArchitectureID string
type SpecID string
type InterventionID string
type PathID string
type BuildKey string
type ReleaseID string
type SuiteID string
type FidelityID string
type EvaluatorID string
type PolicyID string
type CampaignID string
```

Each identity is domain separated. A `SpecID` cannot be constructed by reusing raw `BuildKey` text. Canonical encodings include schema version and semantic namespace. Human names are metadata, not identity.

D.2 Architecture and specification

```
type Architecture[S any] interface {
    ID() ArchitectureID
    Validate(S) error
    Canonical(S) ([]byte, error)
    BehaviorID(S, ObservationFamily) (SpecID, error)
    DependencyGraph() DependencyGraph
}

type ObservationFamily struct {
    ID        string
    Version   string
    Outcomes  []string
    TraceAxes []string
}
```

An architecture owns validity and semantic dependency meaning. It does not own search strategy or campaign scheduling.

D.3 Typed parameter references

```
type ParamRef[S, A any] interface {
    ID() string
    Lens() Lens[S, A]
    EffectOf(before, after A) Effect
    PrimitiveSupport() NodeSet
    ValidateValue(A) error
}
```

A generated or hand-authored parameter reference combines focus, validation, effect classification, and support. The effect function can be value-sensitive: changing vector mode from exact to approximate has an approximation effect, while changing between two exact algorithms may be operational only under a certified equality relation.

A registry can existentially package typed references behind validated codecs, but the typed constructor should remain the source of truth.

D.4 Atomic intervention

```
type AtomicIntervention[S any] struct {
    ID          InterventionID
    Baseline    SpecID
    Result      SpecID
    Target      string
    Effect      Effect
    Hypothesis  string
    Preserves   []PreservationClaim
    Schema      string
    Apply       func(S) (S, error)
}

type PreservationClaim struct {
    Family    string
    Relation  string
    Tolerance *float64
    Scope     string
}
```

Construction applies the update, validates the result, computes identities, and rejects undeclared changes. Existing `ragopt` one-mutation manifests can instantiate this type.

D.5 Path intervention

```
type Path[S any] struct {
    ID       PathID
    Baseline SpecID
    Result   SpecID
    Steps    []AtomicIntervention[S]
    Targets  NodeSet
    Effect   Effect
}

func Compose[S any](steps ...AtomicIntervention[S]) (Path[S], error)
```

`Compose` verifies adjacent identities, concatenates causal path, unions targets, joins grades, and intersects preservation claims. Its canonical identity includes ordered step identities even if the endpoint equals another path's endpoint.

D.6 Dependency and work planning

```
type DependencyGraph interface {
    Nodes() []Node
    Closure(NodeSet) (NodeSet, error)
    Explain(from NodeSet, to NodeSet) []DependencyPath
}

type ImpactPlan struct {
    Targets          NodeSet
    Closure          NodeSet
    BuildStages      []StageID
    Evaluators       []EvaluatorID
    ReusableArtifacts []ArtifactRef
    ReusableEvidence  []EvidenceRef
    Assumptions      []Assumption
}

type Planner[S any] interface {
    Plan(base S, path Path[S], external ExternalIdentity) (ImpactPlan, error)
}
```

The explanation paths are important operationally. A build request should say not only "rebuild vector index" but "embedding dimension changed, therefore embeddings and vector index are invalidated."

D.7 Reindexing

```
type Reindex[A, B any] interface {
    MapSpec(A) (B, error)
    MapParam(string) (string, error)
    MapSupport(NodeSet) NodeSet
    MapEffect(Effect) Effect
    MapObservation(ObservationFamily) (ObservationFamily, error)
}
```

Composition and identity must satisfy functor laws. A migration adapter may be partial at the construction boundary, but accepted reindexing arrows should be total on their declared subcategory.

D.8 RAG interpreter

```
type ReleaseInterpreter[S any] interface {
    Build(ctx context.Context, spec S, source SourceSnapshot) (BuildArtifact, error)
    Open(ctx context.Context, spec S, build BuildArtifact) (Release, error)
}

type QueryInterpreter interface {
    Direct(ctx context.Context, release Release, req QueryRequest) (EvidenceResult, Trace, error)
    Answer(ctx context.Context, release Release, req AnswerRequest) (AnswerResult, Trace, error)
    Agent(ctx context.Context, release Release, req AgentRequest) (SessionResult, Trace, error)
}
```

Build and open are separated because many candidates share build artifacts while denoting different release behavior. The release is immutable and release-pinned for one request or session epoch.

D.9 Experiment manifest

```
type ExperimentManifest struct {
    ID             Digest
    Baseline       ReleaseID
    Candidate      ReleaseID
    Suite          SuiteID
    Fidelity       FidelityID
    Evaluator      EvaluatorID
    Policy         PolicyID
    SourceBarrier  Digest
    SeedDesign     SeedDesign
    Cases          []CaseID
    Repeats        int
    Supports       map[string]NodeSet
}

type CellKey struct {
    Experiment Digest
    Case       CaseID
    Repeat     int
    Arm        Arm
}
```

The manifest is immutable. A different case set, repeat count, provider, prompt, or censoring rule creates a different experiment identity.

D.10 Evaluator and evidence

```
type Evaluator[Obs any] interface {
    ID() EvaluatorID
    Support() NodeSet
    Evaluate(context.Context, Release, Case, RandomContext) (Obs, error)
    Extract(Obs) ([]MetricDatum, error)
}

type EvidenceCertificate struct {
    Obligation  string
    Candidate   PathID
    Experiment  Digest
```

```
    Artifact    ArtifactRef
    Verifier    Digest
    VerifiedAt  time.Time
    Outcome     string
}
```

The evaluator declares support; the planner decides whether evidence may be reused. The evaluator does not decide promotion.

D.11 Mergeable metric

```
type Reducer[Datum, Summary any] interface {
    Zero() Summary
    Add(Summary, Datum) Summary
    Merge(Summary, Summary) Summary
    Estimate(Summary) (Estimate, error)
}

type Estimate struct {
    Mean   float64
    Low    float64
    High   float64
    N      int
    Method string
}
```

Implementations should provide property tests for associativity, commutativity where claimed, identity, and equivalence between one-pass and partitioned reduction.

D.12 Decision policy

```
type Verdict string
const (
    Pass Verdict = "pass"
    Fail Verdict = "fail"
    NeedMore Verdict = "need-more"
)

type GatePolicy struct {
    ID              PolicyID
    RequiredEffects map[Effect]EvidenceRequirement
    Absolute        []AbsoluteConstraint
    Protected       []NonInferiorityConstraint
    Target          TargetConstraint
}

type DecisionArtifact struct {
    Candidate      PathID
    EvidenceDigest Digest
    Policy         PolicyID
    Verdict        Verdict
    Checks         []Check
}
```

`Evaluate` is pure and deterministic. It consumes a closed evidence bundle and returns a content-identified decision artifact.

D.13 Campaign commands and events

```
type Command interface{ isCommand() }
type Event interface {
    Sequence() uint64
    Campaign() CampaignID
    Digest() Digest
}

type Reducer interface {
    Apply(State, Event) (State, error)
    Check(State) error
}
```

The reducer accepts no search callbacks. Search runs in an outer shell that submits commands. This is the authority boundary.

D.14 Search policy

```
type SearchPolicy[S any] interface {
    Propose(context.Context, SearchView[S], Budget) ([]Path[S], error)
    Allocate(context.Context, AllocationView, Budget) ([]WorkRequest, error)
}
```

The view contains immutable summaries and identities, not raw mutation access to campaign state. Search may use random, Bayesian, evolutionary, gradient-like, bandit, or human-guided methods. All proposals are revalidated by the field adapter and all work flows through the campaign reducer.

D.15 Package ownership

The recommended ownership boundary is:

- `evidencekit` or an equivalent small kernel: canonical identity, immutable references, reducer utilities, law-test helpers;
- `ragkit`: RAG specifications, interpreters, dependency nodes, observation families, release behavior;
- `ragopt`: generic campaign, experiment custody, metric reduction, gates, reports, search interfaces;
- `ragopt/ragspace`: typed RAG interventions, effect doctrine, impact planning, RAG fidelities;
- applications: suites, product metrics, security policy, provider behavior, user/session outcomes, final promotion ownership.

The sandbox combines some of these for legibility. Production packages should preserve the dependency direction even when deployed in one process.

Appendix E. Current-to-target mapping for `ragopt` and `ragkit`

This mapping is based on static inspection of the supplied repositories. The measurements are descriptive, not a substitute for runtime validation.

E.1 Repository scale

The inspected `ragopt` snapshot contains 45 Go files, approximately 5,925 nonblank Go lines, 42 test functions, and 12 package directories. The inspected `ragkit` snapshot contains 173 Go files, approximately 17,743 nonblank Go lines, 273 test functions, and 23 package directories. The sandbox contains 42 Go files, approximately 3,743 nonblank Go lines, 29 test functions, and 15 package directories.

The important conclusion is qualitative: both existing packages already contain meaningful kernels. The target architecture should evolve them rather than replace them with a speculative universal framework.

E.2 `ragopt` assets to preserve

The current optimizer's strongest semantics are immutable custody and exact comparison discipline. These should remain stable:

1. **Immutable snapshots.** Reinterpret them as objects of a fiber and as endpoints of atomic generators.
2. **Exactly one declared mutation.** Preserve as the constructor rule for atomic interventions.
3. **Copied asset verification.** Incorporate artifact and external identities into evidence certificates.
4. **Exact paired cells.** Generalize the cell key with fidelity, evaluator, policy, and seed-design identities.
5. **Explicit missingness.** Retain missing pairs as first-class decision input.
6. **Ordered gates.** Extend with interval non-inferiority and effect-derived obligations; do not replace with scalar search score.
7. **Durable run directories and resume.** Place behind the event reducer and work/outbox protocol.
8. **Reports.** Extend with path, effect, closure, reuse explanation, evidence certificates, and feasible Pareto frontier.

E.3 `ragopt` additions

The smallest coherent additions are:

- `intervention`: versioned atomic and path manifests;
- `effect`: generic grading interfaces, with RAG grades in an adapter;
- `experiment`: experiment/fidelity identity and randomization design;
- `campaign`: append-only authority reducer;
- `support`: generic support sets and cache-admission predicates;
- `pareto`: feasible frontier reporting;
- `ragspace`: RAG-specific parameter references, dependency doctrine, and obligations.

A generic `ragopt` package should not define chunkers, vector index settings, answer traces, or widget semantics. Those belong to imported adapters.

E.4 `ragkit` assets to preserve

The existing package boundaries already expose core RAG meanings: chunking, representation, embedding, indexes, retrieval, reranking, answering/generation, and evaluation. They can become the semantic nodes and interpreters of the field.

Preserve:

- deterministic evidence and chunk identity rules;
- total ranking and tie-break laws;
- explicit retrieval stages rather than one opaque query function;
- native result and trace types;
- backend capabilities and approximate-search configuration;
- evaluation types that are genuinely common across products.

E.5 `ragkit` additions

Add one behavior-complete optimization-facing specification assembled from existing domain types. It should bind:

- source and corpus identity;
- derivation/chunk/representation specifications;
- lexical and vector backend specifications;
- query rewrite, filtering, fusion, reranking, and context policies;
- answer/agent and provider identities where shared;
- serving, timeout, fallback, and projection policies where they affect common observations.

Then add:

- typed parameter references or generated lenses;
- a versioned dependency schema;
- build-key projections;
- observation-family declarations;
- exact-oracle hooks for approximation validation;
- interpreters that expose native traces and release identities.

The aggregate specification should not become a second implementation of every subsystem. It is a manifest of identities and references to native specifications.

E.6 Overlap elimination

When GEC, RAG-TTC, or Garden contains a copied common substrate, migration should use differential fixtures:

1. capture inputs, release identities, and native outputs from the current implementation;
2. interpret the equivalent `ragkit` specification;
3. compare total ranking, evidence identity, trace projection, and errors;
4. characterize intentional differences as typed interventions;
5. cut over only after the declared observation relations pass.

This avoids calling a rewrite “semantics-preserving” based only on final answer text.

E.7 Application ownership

GEC retains authorization, source scope, synonyms, provider disclosure policy, judges, and admin behavior. RAG-TTC retains product catalogs, connected retrieval, tool loops, provider integration, review, and tool evaluation. Garden retains multi-turn intent, choices, widget semantics, product facts, calibration, and frontend session behavior.

The shared field can express interventions into these domains, but it does not own their validity. Applications register additional effect grades, obligations, observation families, and gates through monotone extensions.

E.8 Compatibility strategy

Version every semantic surface independently:

- architecture schema;
- specification schema;
- intervention schema;
- dependency graph;
- effect doctrine;
- experiment manifest;
- evaluator;
- metric reducer;
- decision policy;
- campaign reducer.

A candidate is valid only under a closed version vector. Compatibility code should translate old artifacts into new types explicitly rather than allowing implicit default fields to alter meaning.

Appendix F. Sandbox experiment protocol and interpretation

F.1 Purpose

The sandbox is a self-contained executable model of the proposed architecture. It uses only the Go standard library, requires no model provider or database, and runs deterministically from a fixed seed design. Its role is to answer a concrete question: can the algebra support a joint indexing/query optimization campaign with lawful composition, dependency-aware reuse, multi-fidelity allocation, uncertainty-aware promotion, and runtime authority?

It is not a benchmark of modern embedding models or a production latency study.

F.2 Toy corpus and queries

The corpus contains short documents with overlapping concepts and controlled lexical variation. The tokenizer applies deterministic normalization and a small semantic token map to emulate representational similarity without external embeddings. Query cases carry relevance judgments and answer-support terms.

The design intentionally creates tension among lexical retrieval, vector retrieval, chunk boundaries, candidate depth, reranking, context size, latency, and build cost. A trivial corpus where every candidate ties would not exercise the architecture.

F.3 Behavior-complete candidate specification

The candidate specification includes:

- chunk word limit;
- chunk overlap;
- embedding dimension;
- vector mode (`exact` or modeled `approximate`);
- candidate depth;
- fusion weight;
- reranking flag;
- context size.

The build-key projection includes only chunking, overlap, dimension, and vector-mode inputs that affect build artifacts. The full behavior identity includes all eight fields.

Candidate enumeration uses finite typed and dependent spaces. Invalid combinations are filtered before evaluation. The baseline is excluded from the 575 non-baseline candidates.

F.4 Dependency graph

Primitive parameters flow into these representative nodes:

- `artifact.chunks`;
- `artifact.lexical_index`;
- `artifact.embeddings`;
- `artifact.vector_index`;

- query lexical and vector channels;
- candidate generation;
- fusion;
- reranking;
- context admission;
- retrieval, answer, and operations evaluators.

A change to fusion weight reaches fusion, downstream ranking, context, and all dependent evaluators but not build artifacts. A change to chunk size reaches every build artifact and downstream observation. The recommended candidate changes query-time fields only, so its build key equals the baseline's and the build is reused.

F.5 Query interpreter

For each query, the interpreter:

1. tokenizes the query;
2. scores chunks lexically;
3. computes deterministic vector scores;
4. optionally applies approximate vector selection;
5. takes per-channel candidate prefixes;
6. fuses channels by a weighted score;
7. optionally reranks with a deterministic support feature;
8. admits the top context items;
9. produces retrieval and answer-support observations;
10. records modeled latency and build work.

All rankings use deterministic tie breaks. Approximation is compared to an exact-vector oracle metric.

F.6 Metrics

The campaign records:

- mean reciprocal rank (`mrr`);
- recall at admitted context (`recall_at_context`);
- answer support (`answer_support`);
- exact-oracle agreement (`exact_oracle_agreement`);
- security compliance sentinel (`security_compliance`);
- modeled query latency (`latency_ms`);
- modeled build work (`build_units`).

Every paired metric retains baseline, candidate, delta, sample count, and interval. The security metric demonstrates gate plumbing only; it is not a security test.

F.7 Fidelity chain

The chain is:

- `mini`: six cases, one repeat;
- `full`: twelve cases, three repeats;
- `final`: twelve cases, sixteen repeats.

The chain is validated by increasing rank and cost. It is used as a budget-allocation policy. The thesis does not assert a Blackwell relation among these three empirical fidelities. A separate exact finite example demonstrates the garbling checker.

F.8 Allocation and promotion

All 575 candidates run at mini fidelity. A scalar heuristic allocates 56 candidates to full fidelity and 14 to final fidelity. The scalar combines quality and modeled cost only to spend the evaluation budget. It has no promotion authority.

The final policy requires:

- answer support lower bound at least 0.68;
- latency upper bound at most 12.5;
- oracle-agreement lower bound at least 0.70;
- security compliance lower bound at least 1;
- recall non-inferiority margin 0.04;
- answer-support non-inferiority margin 0.03;
- MRR improvement lower bound at least 0.005.

Ordered gates return `need-more` when an interval crosses a threshold.

F.9 Results

The campaign identifies six final candidates that both pass the gate and lie on the feasible Pareto frontier. The selected release is:

```
chunk_words      18
chunk_overlap     0
embedding_dim    16
vector_mode       exact
candidate_depth   6
fusion_alpha      0.55
rerank             true
context_k          3
```

Relative to baseline at final fidelity:

- MRR increases from approximately 0.9036 to 1.0000;
- recall increases from approximately 0.9427 to 1.0000;
- answer support increases from approximately 0.4378 to 0.7472;
- exact-oracle agreement increases from approximately 0.8737 to 1.0000;
- modeled latency increases from approximately 5.8365 to 12.1103;
- modeled build work remains approximately 788.6.

The candidate's effect grade is approximation plus relevance plus interaction. It requires no rebuild because its indexing parameters equal baseline. Its final decision passes all obligations and gates. The event reducer records proposal, required work, completed evaluation, decision, and promotion.

F.10 Cache evidence

The recorded build-cache hit rate is approximately $99.9\,\%$ because many candidate endpoints share one of a small number of build identities. The query/evaluation cache hit rate is approximately $54.0\,\%$ under support-aware reuse in the finite campaign.

These percentages are properties of the enumerated toy space, not forecasts for production. Their significance is structural: build sharing is determined by build-key projection, and evaluator reuse is determined by impact/support disjointness plus experiment identity.

F.11 State-space exploration

The Go model checker explores reducer states to depth nine. The report contains 477 distinct sequence-sensitive states and 1,496 accepted transitions. It checks reducer invariants after every accepted transition and verifies that illegal actions are rejected.

Sequence number is included in the canonical state key, so states with identical candidate status but different event counts remain distinct. This makes the count unsuitable as a minimal quotient-state measure but appropriate for auditing the implemented reducer exploration.

F.12 Reproduction

From the sandbox root:

```
./scripts/reproduce.sh
```

The script creates the demo report, JSONL event stream, candidate and frontier CSVs, law report, model-check report, dependency graph, and checksums. `campaign.json` is large because it retains full paired observations rather than only summaries.

F.13 Interpretation boundary

The campaign establishes that:

- the finite category laws hold for tested examples;
- lenses satisfy their finite law suite;
- change closure drives build and evaluator reuse;
- paths retain effect and causal identity;
- exact pairing and interval gates execute end to end;
- search score cannot bypass the reducer;
- a joint indexing/query field can be enumerated and evaluated.

It does not establish external validity for real corpora, provider behavior, human answer quality, security, or production latency. Those require application-specific experiments and operational evidence.

Appendix G. Verification matrix and formal-method plan

G.1 Evidence levels

The project uses four evidence levels:

1. **Mathematical proof:** a theorem follows from stated axioms.
2. **Mechanized finite proof or exhaustive exploration:** all states or values of a bounded finite model are checked.
3. **Property and differential testing:** many generated or fixture-backed cases are checked.
4. **Production observation:** behavior is measured under real workloads, providers, failures, and user populations.

These levels are complementary. A proved closure theorem is useless if the production graph omits an edge. A load test cannot prove promotion is unreachable without pass in every event order. The architecture assigns each claim to an appropriate level.

G.2 Kernel verification matrix

Canonical identity. Test determinism, map-order independence, domain separation, schema-version sensitivity, and collision handling. Production uses cryptographic digests; proof assumes collision resistance.

Finite kernels. Property-test normalization, identity, associativity, tensor bifunctoriality, copy/discard laws on deterministic data, and total-variation symmetry. Extend with rational arithmetic for exact small proofs if numerical tolerance becomes material.

Lenses. Exhaustively check get-put, put-get, and put-put over finite enum fields. Generate lenses from schema metadata where possible. Mutation-test `Put` to ensure the suite catches unrelated-field drift.

Change actions. Check action identity and associativity, then derivative squares over finite models. For production build stages, use differential fixtures: incremental maintenance must equal clean rebuild at the same source barrier.

Dependency graph. Check acyclicity or explicitly model feedback SCCs; test closure laws; compare predicted invalidation with instrumented dataflow; use mutation testing to delete edges and verify differential tests fail.

Effect doctrine. Exhaustively check grade join laws and monotonicity of required evidence. Reject unknown effect bits. Product extensions must prove their forgetful map is monotone.

Reindexing. Check identity and composition laws, preservation of valid specifications, and support transport. Migration fixtures should compare denotations under declared observation families.

Metric reducers. Property-test identity, associativity, commutativity when claimed, partition independence, and numerical tolerance. Compare summaries against retained raw observations.

Decision kernel. Exhaustively test threshold boundaries, missing metrics, incomplete pairs, need-more intervals, policy versioning, and phase order. Mutation-test phase order to ensure target improvement cannot override safety.

G.3 Campaign model checking

The included TLA+ module models proposal, build, evaluation, decision, promotion, and rejection. Its principal safety properties are:

- no promotion without pass;
- no candidate both promoted and rejected;
- no evaluation before a required build;
- no completion before start;
- active implies promoted.

A production model should add:

- multiple candidates and fidelities;
- retry attempts versus semantic cells;
- durable outbox delivery;
- activation and rollback;
- release leases;
- sequence compare-and-swap;
- cancellation races;
- policy upgrade and evidence invalidation.

The bundle states explicitly that TLC was not executed in the build environment. The Go bounded explorer was executed. Running TLC is a recommended next verification step, not retroactive evidence.

G.4 Differential oracles

The strongest practical oracles for RAG optimization are:

- clean full rebuild versus incremental build;
- exact vector retrieval versus approximate backend;
- current product pipeline versus shared `ragkit` interpreter;
- direct one-pass reducer versus partitioned/merged reducer;
- online materialized campaign state versus log replay;
- frontend full replay versus snapshot-plus-suffix projection;
- baseline/candidate cells reconstructed from retained native artifacts versus stored summaries.

Every oracle comparison names an equality or tolerance relation. “Looks similar” is not a test relation.

G.5 Fuzzing

Fuzz these boundaries:

- canonical encoding of nested specifications;
- malformed intervention paths and mismatched adjacent IDs;
- graph node/edge insertion and closure;
- event sequences including duplicates and reorderings;
- experiment cells with missing, duplicated, or inconsistent identities;
- metric NaN, infinity, zero count, and extreme values;
- policy thresholds and interval degeneracy;
- query cancellation and partial trace projection.

Fuzzers should preserve failed inputs as regression fixtures with semantic names.

G.6 Fault injection

Inject failures at every durable boundary:

- after command validation but before event append;
- after event append but before materialization;
- after outbox append but before worker submission;
- during build staging;
- after artifact write but before verification;
- after one evaluation arm completes;
- during metric merge;
- after decision persistence but before promotion request;
- during activation and rollback.

The desired invariant is not “nothing fails.” It is that every failure yields an explicit, replayable state with no unauthorized promotion or stale artifact reuse.

G.7 Statistical validation

The sandbox uses normal intervals for simplicity. Production should select methods per metric and design:

- paired bootstrap or permutation intervals for bounded ranking metrics;
- stratified resampling for heterogeneous case groups;
- cluster-robust methods for repeated conversation or user units;
- sequentially valid intervals for adaptive online stopping;
- multiplicity control or hierarchical policies when many candidates share one campaign;
- predeclared censoring and missingness treatment.

The statistical method identity belongs in the estimate and policy artifact. Recomputing with a different method produces a new evidence bundle.

G.8 Security verification

Policy/security interventions require independent treatment. At minimum:

- authorization precedes hydration and remote disclosure;
- provider payloads are derived only from authorized evidence;
- traces record disclosure identities without leaking protected content;
- evaluator and optimizer principals have least authority;
- candidate manifests cannot smuggle provider or prompt changes through untyped extension fields;
- promotion and activation require distinct capabilities;
- native evidence artifacts have access controls and retention policy.

A security sentinel metric is never a substitute for these controls.

G.9 Production acceptance criteria

A first production adoption should require:

1. every released candidate has an immutable path manifest and behavior identity;
2. impact plans explain every rebuild and reuse decision;
3. clean rebuild differential tests pass for indexing changes;
4. approximation changes pass an exact-oracle suite;

5. paired cells are complete or explicitly missing;
6. all required obligations have native artifact certificates;
7. the decision artifact is reproducible from retained evidence;
8. the reducer prevents promotion without pass under model checking and tests;
9. activation is release-pinned and rollbackable;
10. monitoring can attribute production observations to release and experiment identity.

Appendix H. Staged implementation and migration program

The migration is designed to create value before the full theory is implemented. Each stage has an exit criterion and preserves compatibility with existing campaigns.

H.1 Stage 0: freeze semantic vocabulary

Document current candidate, snapshot, build, evaluation, gate, and run identities in `ragopt`; document chunk, representation, index, query, context, answer, and trace identities in `ragkit`. Mark which fields can change behavior and which are operational metadata.

Exit criterion: two maintainers can independently compute the same identity and say whether a field change requires a new release.

H.2 Stage 1: typed atomic descriptors

Wrap current one-mutation candidates in a versioned descriptor containing target parameter, before/after identity, effect grade, hypothesis, and preservation claims. Do not add path search yet.

Recompute the descriptor from copied snapshots and reject mismatches. Add law tests for grade monotonicity and hidden mutation detection.

Exit criterion: every existing campaign can run through the descriptor adapter without changing its final report, and every candidate has a machine-readable semantic target.

H.3 Stage 2: dependency graph and build keys

Define the first RAG dependency graph around stable stages: source, chunk, representations, lexical index, vector index, retrieval channels, fusion, reranking, context, answer, and evaluators. Add build-key projection and an explainable closure API.

Initially use conservative edges. Instrument actual stage inputs and compare predicted closure with clean rebuilds and current pipeline behavior.

Exit criterion: query-only candidates demonstrably reuse build artifacts, and every indexing candidate that requires rebuild is detected by differential tests.

H.4 Stage 3: path candidates

Add ordered path manifests over atomic candidates. Verify adjacent snapshots and preserve path identity. Introduce generated finite spaces and dependent filters as search front ends, but keep existing grid/search implementations working.

Do not infer parallel independence automatically. Add a certificate interface and use it only for well-understood disjoint changes.

Exit criterion: a two- or three-factor coordinated experiment executes without opaque multi-mutation configuration files, and endpoint-equivalent paths remain distinguishable in reports.

H.5 Stage 4: experiment identity and support-aware reuse

Version experiment manifests with suite, case set, repeats, seed design, fidelity, evaluator, source barrier, and policy. Require evaluators to declare support. Admit cached observations only when both semantic identity and support-disjointness pass.

Retain all native cells and explicit missingness. Introduce mergeable metric reducers with partition-law tests.

Exit criterion: an evaluator reuse report explains why each artifact was reused or recomputed, and replay from native cells reproduces every summary.

H.6 Stage 5: evidence doctrine and interval gates

Map effect grades to evidence obligations. Attach native certificates. Extend gates with interval-valued absolute and non-inferiority checks. Preserve the existing ordered-gate behavior for legacy policies.

Add feasible Pareto reporting after gates. Keep the final selection explicit and product-owned.

Exit criterion: no candidate can pass with an unmet obligation, incomplete required pairing, or interval crossing a hard threshold.

H.7 Stage 6: campaign reducer and job integration

Put campaign authority behind the append-only reducer. Connect builds and evaluations through an outbox and idempotent job identities. Treat search as an untrusted command producer.

Model-check the event protocol and run fault-injection tests. Separate campaign promotion authorization from release activation.

Exit criterion: crash/replay and duplicate-delivery tests preserve state; promotion without pass is unreachable in the checked model and rejected in implementation.

H.8 Stage 7: product fibers

Implement adapters for at least two applied systems before generalizing further.

For GEC, focus first on authorization/disclosure ordering, synonyms, reranking, judges, and admin traces. For RAG-TTC, focus on complete builds, ANN certification, connected retrieval, tool trajectories, and review. For Garden, focus on session epochs, product facts, choices, widgets, and frontend convergence.

Promote only abstractions that retain the same laws and meanings in at least two fibers. Similar names are insufficient.

Exit criterion: shared campaigns can compare common RAG interventions while product-specific effects and gates remain native.

H.9 Stage 8: online feedback

Add canary and production-observation fibers with traffic, temporal, user, and privacy identities. Use immutable releases and routing interventions. Introduce sequentially valid decisions where appropriate.

Keep online data from silently rewriting offline suites or active release parameters. Feedback creates new candidate hypotheses.

Exit criterion: every online observation is attributable to one release epoch and experiment protocol, and rollback does not mix evidence across releases.

H.10 Governance

Create architectural decision records for:

- behavior-complete release identity;
- dependency graph ownership;
- effect taxonomy and extension;
- evidence certificate schemas;
- experiment and fidelity identity;
- gate policy authority;
- campaign event schemas;
- activation capability separation.

Schema evolution requires migration tests and explicit compatibility statements. Mathematical terminology should appear in design documents where it clarifies laws; production APIs should use domain language familiar to engineers.

H.11 First three implementation projects

A practical first quarter can be organized as three self-contained projects.

Project A: Closure and build reuse. Implement typed parameter references, a RAG dependency graph, build-key projection, and differential clean-rebuild tests. Demonstrate reuse for fusion/reranking and rebuild for chunk/embedding changes.

Project B: Paired evidence and gates. Extend experiment identity, support declarations, interval estimates, non-inferiority, and obligation certificates. Reproduce one existing campaign through both old and new reports.

Project C: Campaign authority. Implement event schemas, reducer, replay, outbox integration, and model checking. Keep the existing search policy and workers unchanged behind adapters.

These projects can proceed partly in parallel once the shared identity schema is fixed.

Appendix I. Bibliography

Alvarez-Picallo, Mario, and C.-H. Luke Ong. 2019. “Change Actions: Models of Generalised Differentiation.” In *Foundations of Software Science and Computation Structures (FoSSaCS 2019)*, 45–61. Lecture Notes in Computer Science. Springer. DOI: 10.1007/978-3-030-17127-8_3.

Blackwell, David. 1953. “Equivalent Comparisons of Experiments.” *Annals of Mathematical Statistics* 24(2): 265–272.

Capucci, Matteo, Bruno Gavranović, Jules Hedges, and Eigil Fjeldgren Rischel. 2022. “Towards Foundations of Categorical Cybernetics.” *Electronic Proceedings in Theoretical Computer Science* 372: 235–248. DOI: 10.4204/EPTCS.372.17. arXiv:2105.06332.

Cruttwell, Geoffrey S. H., Bruno Gavranović, Neil Ghani, Paul Wilson, and Fabio Zanasi. 2022. “Categorical Foundations of Gradient-Based Learning.” In *Programming Languages and Systems: ESOP 2022*, 1–28. Lecture Notes in Computer Science 13240. Springer. DOI: 10.1007/978-3-030-99336-8_1.

Fong, Brendan, and David I. Spivak. 2019. *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge University Press.

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239. DOI: 10.1016/j.aim.2020.107239.

Fritz, Tobias, Tomáš Gonda, Paolo Perrone, and Eigil Fjeldgren Rischel. 2023. “Representable Markov Categories and Comparison of Statistical Experiments in Categorical Probability.” *Theoretical Computer Science* 961: 113896. DOI: 10.1016/j.tcs.2023.113896. arXiv:2010.07416.

Jacobs, Bart. 1999. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics 141. Elsevier.

Kleisli, Heinrich. 1965. “Every Standard Construction Is Induced by a Pair of Adjoint Functors.” *Proceedings of the American Mathematical Society* 16(3): 544–546.

Libkind, Sophie, Andrew Baas, Evan Patterson, and James Fairbanks. 2022. “Operadic Modeling of Dynamical Systems: Mathematics and Computation.” *Electronic Proceedings in Theoretical Computer Science* 372: 192–206. DOI: 10.4204/EPTCS.372.14. arXiv:2105.12282.

Mac Lane, Saunders. 1998. *Categories for the Working Mathematician*. Second edition. Graduate Texts in Mathematics 5. Springer.

Marcolli, Matilde. 2022. “Pareto Optimization in Categories.” arXiv:2204.11931.

Riley, Mitchell. 2018. “Categories of Optics.” arXiv:1809.00738.

Spivak, David I. 2014. *Category Theory for the Sciences*. MIT Press.

Vagner, Dmitry, David I. Spivak, and Eugene Lerman. 2015. “Algebras of Open Dynamical Systems on the Operad of Wiring Diagrams.” *Theory and Applications of Categories* 30: 1793–1822.

Repository and artifact sources

The empirical sections use the supplied August 2026 source snapshots of `ragkit`, `ragopt`, GEC, RAG-TTC, and TTC Garden, together with the preceding volume *The Semantics and Dynamics of Retrieval-Augmented Systems*. Repository measurements in this volume were computed by deterministic local

scripts and describe only the inspected snapshots. The sandbox source, complete campaign artifacts, figures, checksums, and QA reports are included in the source bundle.

Appendix J. Glossary

Absolute constraint. A one-sided requirement on a candidate estimate, such as a latency upper bound or security lower bound. It is checked before target improvement.

Architecture. The typed component and wiring structure of a RAG system, including the semantic nodes over which dependency and observation doctrines are defined.

Architecture mapping. A restriction, embedding, migration, or translation between architectures that induces reindexing between optimization fibers.

Artifact support. The semantic node set through which an artifact's denotation factors. It is stronger than a list of files accessed during execution.

Atomic intervention. One validated, typed change to a behavior-complete specification. It is a generator in the intervention category.

Behavior-complete specification. A release specification containing every identity capable of changing a selected family of observations.

Blackwell order. A preorder on statistical experiments. H is at least as informative as L when L can be obtained by garbling H.

Build key. The projection of a full release specification onto parameters that determine build artifacts.

Candidate. A morphism from a baseline specification, including causal path, effect grade, targets, and endpoint. It is not merely an endpoint configuration.

Campaign. The runtime process that proposes, builds, evaluates, decides, and authorizes promotion of candidates under an immutable protocol.

Campaign reducer. The small partial state transition function that is authoritative for campaign status and promotion safety.

Change action. A monoid of changes acting on values, used to define generalized derivatives and incremental computation.

Closure. The least downstream-closed node set containing an intervention's primitive targets.

Commutativity certificate. Evidence that two interventions may be reordered or composed in parallel under a declared observation relation.

Coslice category. For baseline θ_0, the category whose objects are arrows out of θ_0 and whose morphisms extend those arrows. It models progressive candidates.

Decision artifact. A content-identified record of policy, evidence digest, ordered checks, and pass/fail/need-more verdict.

Denotational semantics. The mathematical behavior assigned to a release or campaign, independent of one operational implementation but retaining selected outputs and traces.

Dependency derivative. A conservative abstraction of how a local change propagates to artifacts, runtime stages, and evaluators.

Effect grade. A join-semilattice element describing which semantic concerns an intervention may alter.

Evidence certificate. A verified reference showing that a specific obligation was discharged for a candidate under explicit identities.

Evidence doctrine. The monotone mapping from effect grades to required evidence, indexed by architecture and product policy.

Experiment. A stochastic observation channel generated from a release under controlled cases, randomization, evaluator, and fidelity.

External identity. A semantic input not represented as a dependency node, such as suite, seed design, provider version, source barrier, or policy.

Feasible candidate. A candidate that passes required evidence, completeness, absolute, and protected gates under the selected policy.

Fidelity. An experiment configuration with a particular information and cost profile. Numeric rank alone does not imply Blackwell comparability.

Fiber. The intervention category belonging to one architecture in the indexed optimization field.

Garbling. A stochastic post-processing channel that turns one experiment's observations into another's.

Grade join. Conservative combination of semantic effects under intervention composition; no concern is lost.

Impact plan. The trusted result of applying dependency closure and work/evidence rules to an intervention.

Incremental rebuild. Maintenance of derived artifacts by applying a change plan rather than performing a clean full build, subject to a differential equivalence oracle.

Intervention. A typed, validated, causal change from one valid specification to another.

Intervention path. An ordered composition of atomic interventions whose adjacent identities verify.

Kleisli composition. Sequential composition of effectful or stochastic functions by binding the output distribution of one into the next.

Lens. A lawful getter/putter pair that focuses and updates a local part of a whole specification.

Markov category. A symmetric monoidal category modeling stochastic maps with copy and discard structure on data objects.

Metric reducer. A mergeable algebra that converts native observations into estimates while supporting partitioned execution.

Need-more. The decision produced when evidence does not establish either side of a policy threshold, commonly because an interval overlaps it.

Non-inferiority. A protected-metric relation requiring the candidate not to be worse than baseline by more than a declared margin.

Observation family. The selected projection of behavior—answers, evidence, traces, disclosure, latency, frontend events, or sessions—used to state equivalence or refinement.

Operational semantics. The labeled transition rules describing how campaign or RAG runtime state evolves step by step.

Optic. A compositional abstraction for focusing through structured systems while retaining residual context and, in general, backward information.

Optimization field. The indexed family $Opt : A^{op} \to Cat$ of intervention categories over architectures, together with effect, dependency, experiment, and decision structure.

Oracle. A reference interpreter or relation used to validate another implementation, such as exact retrieval for approximate search or clean rebuild for incremental maintenance.

Paired cell. Baseline and candidate observations sharing exactly the same controlled case, repeat, seed design, fidelity, evaluator, and policy identity.

Parameterized morphism. A system arrow $P \otimes X \to Y$ whose parameter object remains explicit under composition.

Pareto frontier. The feasible candidates not dominated across the selected multi-objective metric order.

Path identity. Content identity of the ordered atomic intervention history, not only its endpoint.

Preservation claim. A statement that an intervention retains a named observation relation over a declared scope and tolerance.

Promotion. Campaign authorization that a candidate has passed the declared evidence policy. Deployment or traffic activation may be a separate protocol.

Reindexing. Transport of specifications, interventions, support, and observations along an architecture mapping.

Release. An immutable, behavior-complete interpretation unit against which queries and experiments are pinned.

Replay adequacy. Equality between campaign state reconstructed from the durable event log and authoritative online reduction of the same prefix.

Reuse. Acceptance of an existing artifact or observation for a candidate after support-disjointness and external-identity checks.

Search policy. An untrusted strategy that proposes candidates and allocates budget. It has no promotion authority.

Semantic node. A parameter, derived artifact, runtime stage, observation, or evaluator in the architecture dependency doctrine.

Stateful stochastic transducer. A stochastic arrow that consumes state and input and returns new state, output, and trace.

Statistical semantics. The experiment and inference layer connecting a release's behavior to finite evidence and decisions under uncertainty.

Support-disjointness. The condition that an intervention's impact closure does not intersect an artifact or evaluator's semantic support.

Tensor. Parallel composition in a monoidal category. In a stochastic setting it represents declared independence unless shared state is explicit.

Trace. Structured intensional events recording lineage, disclosure, fallbacks, latency, cost, tools, streaming, and other behavior not captured by final output alone.

Trusted computing base. The small set of identity, validation, closure, pairing, evidence, gate, and reducer code whose correctness is required for safe promotion.

Work plan. The concrete build, evaluation, and verification tasks derived from an impact plan and campaign protocol.

Appendix K. Artifact inventory and integrity

The publication bundle is organized so that the theory can be inspected independently of the executable model and the executable model can be reproduced independently of the formatted thesis.

The top-level artifacts are:

- the Markdown manuscript;
- the editable DOCX publication;
- the rendered PDF publication;
- the complete `optfield-sandbox` source tree;
- all diagram sources and rendered figures;
- repository measurements and selected campaign projections;
- DOCX accessibility output, page-render metrics, PDF preflight, and checksums.

The sandbox `demo-output` directory contains the complete campaign evidence. Derived CSV and Markdown reports are conveniences. The JSON/JSONL artifacts and immutable source are the primary reproduction inputs.

Every bundle should be verified with its checksum manifest after transfer. A checksum establishes byte integrity, not semantic correctness. Semantic verification still requires running tests, law checks, the model checker, and the campaign under the documented toolchain.